CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài và Tính cấp thiết

Trong bối cảnh bùng nổ của các nền tảng âm nhạc trực tuyến như Spotify, Apple Music hay YouTube Music, nhu cầu tìm kiếm bài hát của người dùng ngày càng đa dạng. Tuy nhiên, có một thực tế là:

- Người dùng thường xuyên rơi vào tình huống **"lờ mờ về giai điệu"**: Nhớ giai điệu nhưng không nhớ tên bài hát, tên ca sĩ hay lời nhạc.
- Các hệ thống tìm kiếm truyền thống dựa trên văn bản (Metadata Search) hoàn toàn bất lực trong trường hợp này.
- Các hệ thống nhận diện âm thanh qua vân tay (Audio Fingerprinting - như Shazam) yêu cầu đoạn nhạc gốc phải có chất lượng cao và khớp hoàn toàn về tần số, do đó không thể nhận diện được tiếng huýt sáo, tiếng hát (humming) của con người vốn đầy rẫy sai số.

=> Việc xây dựng một hệ thống **Query-by-Humming (QBH)** là giải pháp cầu nối giúp cá nhân hóa trải nghiệm người dùng, biến những giai điệu "vụt qua trong đầu" thành kết quả tìm kiếm chính xác.

1.2. Mục tiêu nghiên cứu

Đề tài tập trung vào việc giải quyết bài toán tìm kiếm âm nhạc dựa trên giai điệu đầu vào từ phía người dùng (tiếng hát hoặc tiếng huýt sáo) với các mục tiêu cụ thể:

1. **Trích xuất đặc trưng bền vững:** Nghiên cứu phương pháp chuyển đổi tín hiệu âm thanh thô sang dạng phổ **Constant-Q Transform (CQT)** để tối ưu hóa đặc trưng nhạc lý.
2. **Xây dựng không gian Vector (Embedding Space):** Sử dụng mạng **CNN** kết hợp với **Deep Metric Learning** để mã hóa âm thanh thành các vector ngắn gọn nhưng mang đầy đủ thông tin về giai điệu.
3. **Tối ưu hóa tìm kiếm:** Đảm bảo hệ thống có khả năng truy vấn thời gian thực trên một cơ sở dữ liệu lớn mà không bị suy giảm độ chính xác.

1.3. Đối tượng và Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các thuật toán xử lý tín hiệu số (DSP), kiến trúc mạng Neural nhân tạo (CNN) và các hàm mất mát trong Metric Learning (Triplet Loss).
- **Phạm vi:** Nghiên cứu tập trung vào khả năng nhận diện giai điệu (Melody-based) thay vì nhận diện lời bài hát (Lyrics-based). Hệ thống được thử nghiệm trên tập dữ liệu gồm các bản nhạc gốc và các bản humming mô phỏng hoặc thực tế.

1.4. Phương pháp tiếp cận (Sự khác biệt kỹ thuật)

Điểm mới của đề tài này so với các phương pháp cũ (như dùng DTW - Dynamic Time Warping) nằm ở hai trụ cột:

1. **CQT (Trụ cột xử lý tín hiệu):** Thay vì nhìn âm thanh dưới góc độ vật lý (Hz), chúng ta nhìn dưới góc độ âm nhạc (Semitones). Điều này giúp hệ thống "thông minh" hơn khi đối mặt với việc hát lệch tông.
2. **Deep Metric Learning (Trụ cột AI):** Thay vì phân loại cứng nhắc, chúng ta học cách "so sánh". Điều này cho phép hệ thống mở rộng kho nhạc lên hàng triệu bài mà không cần huấn luyện lại (Zero-shot learning).

1.5. Khảo sát các công trình liên quan (Related Work)

Bài toán Query-by-Humming (QBH) đã được nghiên cứu từ cuối những năm 1990. Trải qua hơn hai thập kỷ, các phương pháp tiếp cận đã tiến hóa mạnh mẽ để giải quyết thách thức về tốc độ và độ chính xác. Quá trình phát triển này có thể chia thành ba giai đoạn chính:

1.5.1. Giai đoạn 1: Phương pháp so khớp chuỗi thời gian (Time-Series Matching)

Trong giai đoạn đầu, các hệ thống QBH chủ yếu trích xuất đường viền cao độ (Pitch Contour) từ giọng hát và sử dụng các thuật toán so khớp chuỗi.

- **Đại diện tiêu biểu:** Dynamic Time Warping (DTW) và Hidden Markov Model (HMM).
- **Cơ chế:** Thuật toán DTW sẽ "kéo dãn" hoặc "nén" chuỗi nốt nhạc của người hát để tìm độ sai lệch nhỏ nhất so với chuỗi MIDI gốc.
- **Hạn chế:** * **Độ phức tạp tính toán:** Thuật toán DTW có độ phức tạp $\mathcal{O}(N^2)$ (với N là độ dài chuỗi). Việc quét qua hàng triệu bài hát bằng DTW mất rất nhiều thời gian, không phù hợp cho các hệ thống thời gian thực.
    - **Nhạy cảm với nhiễu:** Các thuật toán này chủ yếu dựa trên tín hiệu thô, dễ bị đánh lừa bởi tiếng ồn môi trường hoặc sự không ổn định trong giọng hát (rung giọng, đứt hơi).

1.5.2. Giai đoạn 2: Phương pháp Học máy phân loại (Machine Learning Classification)

Sự bùng nổ của Deep Learning, đặc biệt là Mạng nơ-ron tích chập (CNN), đã thay đổi cách biểu diễn âm thanh. Các tín hiệu được chuyển thành ảnh phổ (Spectrogram) và đưa vào CNN để phân loại.

- **Cơ chế:** Coi mỗi bài hát trong cơ sở dữ liệu là một "nhãn" (class). Mô hình sẽ nhận đầu vào là đoạn Humming và dự đoán xác suất nó thuộc về bài hát nào thông qua lớp Softmax.
- **Hạn chế:**
    - **Vấn đề Open-set (Tập dữ liệu mở):** Mạng phân loại có cấu trúc đầu ra cố định. Nếu hệ thống có 1.000 bài hát, lớp cuối cùng sẽ có 1.000 nơ-ron. Khi muốn thêm bài hát thứ 1.001, toàn bộ mô hình phải được cấu trúc lại và huấn luyện lại từ đầu.
    - **Khó xử lý biến thiên nội lớp (Intra-class Variance):** Giọng humming của nam, nữ, hoặc một người hát quá tệ có đặc trưng vật lý rất khác nhau, khiến mạng phân loại truyền thống khó hội tụ.

1.6.3. Giai đoạn 3: Tiếp cận bằng Deep Metric Learning (Phương pháp đề xuất)

Để khắc phục hoàn toàn những nhược điểm trên, hệ thống của chúng tôi được thiết kế dựa trên xu hướng hiện đại nhất: **Deep Metric Learning kết hợp Constant-Q Transform (CQT)**.

- **Lý do lựa chọn CQT:** Thay vì dùng MFCC hay STFT truyền thống (vốn phù hợp cho nhận dạng giọng nói speech-recognition), CQT căn chỉnh các dải tần số theo thang logarit của âm nhạc. Điều này giúp mô hình bắt được "linh hồn" của giai điệu, tự động chống chịu với việc người hát bị lệch tông (Pitch Invariance).
- **Lý do lựa chọn Deep Metric Learning**: * **Tốc độ:** Bằng cách biến âm thanh thành các Vector Embedding (ví dụ 128 chiều) và kết hợp với thuật toán tìm kiếm hàng xóm gần nhất (như FAISS của Facebook), hệ thống có thể truy vấn trên cơ sở dữ liệu khổng lồ với độ phức tạp giảm xuống.
    - **Khả năng Zero-shot:** Khi hệ thống cập nhật các bài hát mới ra mắt (như một bài hit vừa phát hành), chúng ta chỉ cần trích xuất vector của bài đó và đưa vào Database. Hệ thống mô hình học sâu **không cần phải huấn luyện lại**.

Tiểu kết: Việc khảo sát cho thấy, tuy các phương pháp DTW hay Classification đã đặt nền móng cho hệ thống QBH, nhưng chúng không đáp ứng được yêu cầu về khả năng mở rộng (Scalability) và độ trễ (Latency) trong môi trường ứng dụng thực tế. Sự kết hợp giữa **CQT** (để tối ưu hóa đặc trưng nhạc lý) và **Metric Learning** (để tối ưu hóa không gian tìm kiếm) là một bước tiến tất yếu và mang tính đột phá cho đề tài này.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ XỬ LÝ TÍN HIỆU ĐẦU VÀO

Trong các hệ thống nhận dạng âm thanh nói chung và hệ thống tìm kiếm bằng giai điệu (Query-by-Humming) nói riêng, pha tiền xử lý và trích xuất đặc trưng đóng vai trò quyết định đến hiệu năng của toàn bộ mô hình. Tín hiệu âm thanh thô chứa rất nhiều thông tin dư thừa và nhiễu, do đó cần được chuyển đổi sang một không gian biểu diễn tối ưu hơn trước khi đưa vào các mô hình học sâu (Deep Learning).

2.1. Phân tích đặc trưng âm thanh và hạn chế của tín hiệu thô

Tín hiệu âm thanh kỹ thuật số là một chuỗi các giá trị biên độ (amplitude) thay đổi theo thời gian, được lấy mẫu ở một tần số nhất định (Sample Rate - f_s), ví dụ 16000Hz hoặc 44100Hz. Một đoạn âm thanh 10 giây ở chuẩn 16000Hz sẽ chứa 160.000 điểm dữ liệu. Việc trực tiếp đưa chuỗi tín hiệu thời gian thô (Raw Waveform) này vào các mạng nơ-ron học sâu thường không mang lại hiệu quả cao vì ba lý do chính:

1. **Chiều dữ liệu quá lớn (High Dimensionality):** Sự bùng nổ về số lượng điểm dữ liệu làm tăng độ phức tạp tính toán, đòi hỏi tài nguyên phần cứng lớn và dễ dẫn đến hiện tượng quá khớp (Overfitting).
2. **Tính phi tĩnh (Non-stationary):** Âm nhạc và giọng hát thay đổi liên tục theo thời gian. Tín hiệu thô không thể hiện trực tiếp được cấu trúc tần số (các nốt nhạc) cấu thành nên âm thanh đó tại một thời điểm cụ thể.
3. **Sự dịch chuyển pha (Phase Shift):** Hai đoạn âm thanh có thể nghe y hệt nhau đối với tai người nhưng cấu trúc sóng thô lại hoàn toàn khác biệt chỉ vì độ lệch pha. Điều này gây khó khăn lớn cho mạng nơ-ron trong việc tìm ra các mẫu (patterns) chung.

Do đó, tín hiệu cần được chuyển đổi từ miền thời gian (Time Domain) sang miền tần số thời gian (Time-Frequency Domain) để trích xuất các đặc trưng cốt lõi của giai điệu.

2.2. Hạn chế của STFT và Ưu thế của CQT

2.2.1. Hạn chế của phép biến đổi Fourier ngắn hạn (STFT)

Phép biến đổi STFT (Short-Time Fourier Transform) là phương pháp phổ biến nhất để phân tích phổ âm thanh. STFT chia tín hiệu thành các khung thời gian ngắn (frames) và áp dụng phép biến đổi Fourier nhanh (FFT) trên từng khung. Tuy nhiên, STFT có một nhược điểm chí mạng đối với âm nhạc: **Độ phân giải tần số là tuyến tính**.

Khoảng cách giữa hai bin tần số liên tiếp trong STFT là một hằng số:

$$\Delta f = \frac{f_s}{N}$$

(với N là kích thước cửa sổ FFT). Trong khi đó, sự cảm nhận cao độ của tai người và cấu trúc của các nốt nhạc lại tuân theo thang đo **logarit**.

Hậu quả là:

- Ở dải tần số thấp (nơi tập trung nhiều năng lượng của giọng hát humming), các nốt nhạc nằm rất sát nhau về mặt tần số vật lý (Hz) nên bị "dính" vào nhau trên phổ STFT, khiến mạng CNN không thể phân biệt được nốt Do và nốt Re.
- Ở dải tần số cao, các nốt nhạc cách xa nhau gây lãng phí độ phân giải và tài nguyên biểu diễn.

2.2.2. Ưu thế của Constant-Q Transform (CQT)

Để khắc phục điểm yếu của STFT, hệ thống đề xuất sử dụng phép biến đổi **Constant-Q Transform (CQT).** Thay vì giữ khoảng cách tần số không đổi, CQT giữ cho tỉ lệ giữa tần số trung tâm và băng thông của bin tần số đó là một hằng số Q:

$$Q = \frac{f_k}{\Delta f_k}$$

Tần số trung tâm của bin thứ k trong CQT được tính theo công thức:

$$f_k = f_{min} \cdot 2^{\frac{k}{B}}$$

Trong đó:

- f_{min} là tần số gốc nhỏ nhất cần phân tích.
- B là số lượng bin trong một quãng tám (Octave). Bằng cách chọn $B = 12$, mỗi bin của CQT sẽ tương ứng chính xác với một bán âm (semitone) trên phím đàn piano.

STFT: Máy tính nghe nhạc bằng "Thước kẻ" (Tuyến tính)

STFT đo lường âm thanh một cách rất máy móc. Giống như một cái thước kẻ có các vạch chia đều nhau từng centimet, STFT chia dải tần số thành các "hộp" (bin) bằng nhau về mặt vật lý (ví dụ: mỗi hộp cách nhau 20Hz).

- Hộp 1: 0 - 20Hz
- Hộp 2: 20 - 40Hz
- Hộp 3: 40 - 60Hz...

Vấn đề là gì? Âm nhạc không hoạt động theo thước kẻ thẳng! Trong âm nhạc, khoảng cách giữa nốt Đô thấp (C2 - 65Hz) và nốt Đô thăng (C#2 - 69Hz) chỉ cách nhau có **4Hz.** Trong khi đó, nốt Đô cao (C5 - 523Hz) và nốt Đô thăng (C#5 - 554Hz) lại cách nhau tới **31Hz.**

- Nếu dùng STFT, ở âm trầm (bass), nốt C2 và C#2 bị nhét chung vào một "hộp" (vì hộp rộng 20Hz). Máy tính sẽ bị mù màu ở dải trầm, không phân biệt được hai nốt này.
- Ở dải cao, các nốt lại nằm rải rác qua quá nhiều hộp, gây lãng phí.

CQT: Máy tính nghe nhạc bằng "Bàn phím Piano" (Logarit)

CQT không dùng thước kẻ. Nó chia tần số giống hệt như các phím đàn trên đàn Piano.

Trong CQT, khoảng cách giữa các "hộp" (bin) không cố định bằng Hz, mà **tỉ lệ thuận** với tần số của hộp đó.

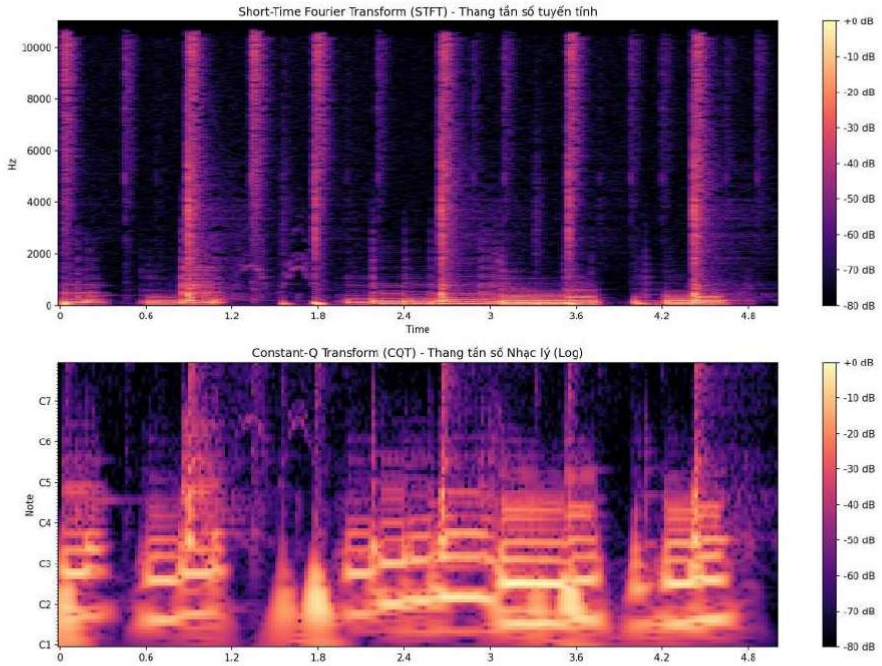
- Ở tần số thấp (âm trầm), các hộp được thu hẹp lại (ví dụ chỉ rộng 2Hz) để hứng trọn vẹn và tách bạch nốt C2 và C#2.
- Ở tần số cao, các hộp được nới rộng ra (ví dụ rộng 40Hz) vì khoảng cách vật lý giữa các nốt ở dải cao vốn dĩ đã xa nhau.

Nhờ vậy, mọi nốt nhạc từ thấp đến cao đều vừa vặn chiếm **đúng một hộp duy nhất.**

Tính chất "Bất biến với cao độ" (Pitch Invariance):

Điểm giá trị nhất của CQT khi kết hợp với Convolutional Neural Network (CNN) là tính bất biến dịch chuyển. Khi một người hát cao hơn hoặc thấp hơn tông gốc (ví dụ hát lệch đi 2 bán âm), toàn bộ cấu trúc giai điệu trên phổ CQT sẽ chỉ đơn giản là trượt lên hoặc trượt xuống theo trục tung (trục nốt nhạc) mà **không bị biến dạng về hình học.** Vì các lớp tích chập (Conv2D) trong mạng CNN sở hữu tính chất bất biến với sự dịch chuyển không gian (Translation Invariance), mạng có thể dễ dàng nhận diện ra giai điệu bất kể người dùng hát ở tông nào.

[Hình 2.1] So sánh ảnh phổ STFT và CQT.



"Ảnh trên: STFT với dải tần số thấp bị nén chặt. Ảnh dưới: CQT với cấu trúc các nốt nhạc được tách bạch rõ ràng theo thang logarit, hiển thị rõ đường nét giai điệu."

2.3. Kỹ thuật lọc nhiễu Spectral Gating

Dữ liệu Humming thực tế thu thập từ người dùng thường được ghi âm qua microphone của điện thoại trong các môi trường không lý tưởng (quán cà phê, đường phố, tiếng quạt gió). Nhiễu nền (Background Noise) làm mờ đi các đặc trưng cao độ, làm giảm độ chính xác của quá trình đối sánh vector. Do đó, kỹ thuật **Spectral Gating** (Cổng phổ) được áp dụng ở pha tiền xử lý nhằm triệt tiêu nhiễu tĩnh mà không làm méo tín hiệu giọng hát.

2.3.1. Nguyên lý hoạt động của Spectral Gating

Khác với các bộ lọc băng thông thông thường (như Low-pass hay High-pass filter) sẽ cắt cụt toàn bộ một dải tần số, Spectral Gating hoạt động một cách linh hoạt dựa trên sự phân bố năng lượng theo thời gian thực. Cơ chế của thuật toán bao gồm 3 bước chính:

Bước 1: Ước lượng biên dạng nhiễu (Noise Profiling)

Hệ thống tự động trích xuất một đoạn âm thanh ngắn (thường là 0.5 giây đầu tiên hoặc các đoạn được xác định là "khoảng lặng" không có tiếng hát) để làm mẫu nhiễu.

Phổ biên độ của đoạn nhiễu được tính toán, sau đó lấy giá trị trung bình $\mu(f)$ và độ lệch chuẩn $\sigma(f)$ của năng lượng tại từng dải tần số f. Tập hợp các giá trị này tạo thành "dấu vân tay" của nhiễu nền.

Bước 2: Xác định ngưỡng cổng (Gate Thresholding)

Đối với mỗi bin tần số f, một ngưỡng giới hạn (Threshold) được thiết lập dựa trên thống kê của nhiễu:

$$T(f) = \mu(f) + \beta \cdot \sigma(f)$$

Trong đó β là hệ số điều chỉnh (thường $\beta \in [1.5, 2.0]$). Ngưỡng này quyết định mức năng lượng tối thiểu mà một tín hiệu tại tần số f phải vượt qua để được coi là "tiếng hát có ích".

Bước 3: Áp dụng mặt nạ phổ (Spectral Masking)

Khi xử lý toàn bộ đoạn âm thanh truy vấn, một mặt nạ (Mask) $M(f,t)$ sẽ được tạo ra cho từng khung thời gian t và tần số f.

- Nếu biên độ của tín hiệu $S(f,t)$ lớn hơn ngưỡng $T(f)$, mặt nạ sẽ mở (giá trị xấp xỉ 1).
- Nếu biên độ nhỏ hơn ngưỡng $T(f)$ (được cho là nhiễu), mặt nạ sẽ đóng (giá trị bị triệt tiêu xuống mức rất nhỏ hoặc bằng 0).

Để tránh hiện tượng xuất hiện tiếng lép bép (Musical Noise) do mặt nạ đóng mở quá gắt, thuật toán áp dụng cơ chế làm mịn (Smoothing) trên cả trục thời gian và trục tần số:

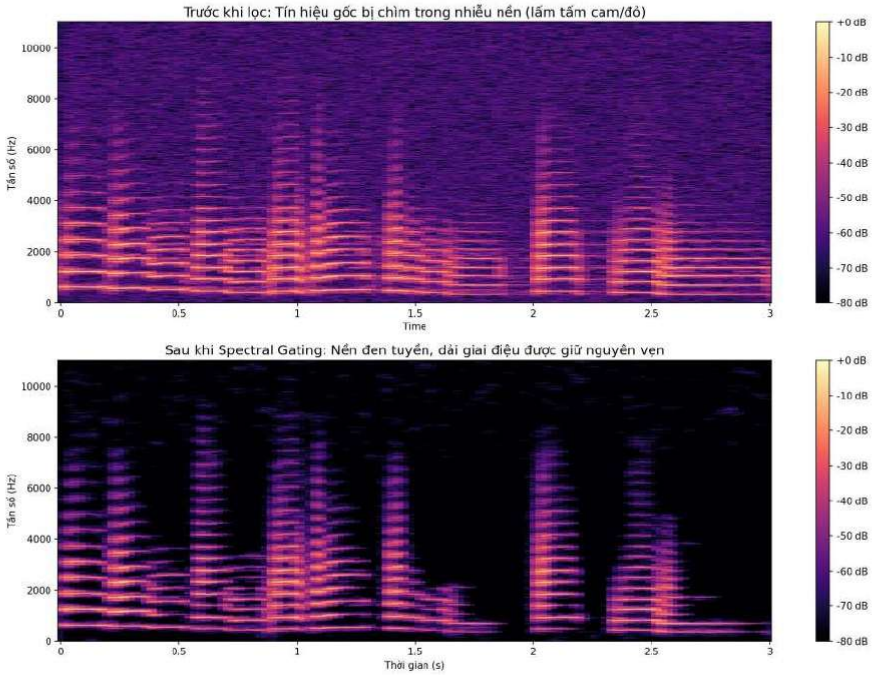
$$S_{clean}(f,t) = S_{noisy}(f,t) \cdot M(f,t)$$

2.3.2. Hiệu quả trên phổ tín hiệu

Sau khi áp dụng Spectral Gating, phổ âm thanh được "rửa sạch" đáng kể. Các nhiễu dải rộng (Broadband noise) hiển thị như những hạt lấm tấm mờ trên Spectrogram bị xóa bỏ, để

lại phần nền đen tuyền (năng lượng bằng 0). Trong khi đó, năng lượng chính của giọng hát (nằm ở các sóng hài - Harmonics) vọt lên trên ngưỡng nhiễu nên được bảo toàn nguyên vẹn sắc thái. Điều này giúp mạng nơ-ron học sâu (CNN) ở các bước sau chỉ cần tập trung vào việc học "hình dáng" của giai điệu mà không bị phân tâm bởi các điểm ảnh nhiễu ngẫu nhiên.

[Hình 2.2] Tín hiệu trước và sau khi Spectral Gating.



Ảnh 1 là bản humming gốc có nhiễu lấm tấm màu cam/đỏ mờ ở nền. Ảnh 2 là bản sau khi lọc với nền hoàn toàn màu đen/xanh đậm, chỉ còn lại các vạch màu vàng/sáng nét căng của giọng hát

CHƯƠNG 3: KIẾN TRÚC MẠNG CNN VÀ TRIPLET LOSS

Sau khi tín hiệu âm thanh được tiền xử lý và chuyển đổi thành ảnh phổ Constant-Q Transform (CQT) ở Chương 2, bài toán Query-by-Humming (QBH) giờ đây được đưa về dạng bài toán xử lý ảnh. Mục tiêu của hệ thống là trích xuất các đặc trưng hình học của dải giai điệu trên phổ CQT và so khớp chúng. Chương này trình bày chi tiết kiến trúc mạng nơ-ron tích chập (CNN) đóng vai trò làm bộ mã hóa (Encoder) và cơ chế hàm mất mát Triplet Margin Loss dùng để tối ưu hóa không gian đặc trưng.

3.1. Thiết kế bộ mã hóa đặc trưng (CNN Encoder)

Do bản chất của âm thanh Humming thường có độ dài thời gian không cố định (người dùng có thể hát 3 giây, 5 giây hoặc 10 giây), việc sử dụng các mạng nơ-ron truyền thống với lớp kết nối đầy đủ (Fully Connected Layer) ở cuối sẽ gặp khó khăn vì chúng yêu cầu đầu vào phải có kích thước cố định. Do đó, hệ thống đề xuất sử dụng kiến trúc mạng CNN thuần túy (Fully Convolutional Network) kết hợp với cơ chế Global Average Pooling.

Cấu trúc chi tiết của mạng bao gồm các khối chính sau:

1. Cụm 3 khối Tích chập cơ bản (3 Conv Blocks)

Mạng sử dụng hàm conv_block để lặp lại một cấu trúc chuẩn hóa gồm 4 lớp con: Conv2d $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool2d.

- **Block 1:** Nhận ảnh gốc (1 kênh màu). Dùng 32 bộ lọc (filters) kích thước 3×3. Qua MaxPool 2×2, kích thước ảnh bị giảm đi một nửa (từ 12×256 xuống 6×128).
- **Block 2:** Nhận 32 bản đồ đặc trưng từ Block 1, tăng lên 64 bộ lọc 3×3. Kích thước tiếp tục giảm còn 3×64.
- **Block 3:** Tăng lên 128 bộ lọc. Kích thước giảm còn 1×32. Lúc này, trục tần số (12 nốt) đã bị nén lại chỉ còn 1, mạng đã tổng hợp được đặc trưng của toàn bộ các nốt.

2. Lớp Tích chập cục bộ (Pointwise Convolution - Conv 1x1)

- **nn.Conv2d(128, 256, kernel_size=1):** Đây là một kỹ thuật rất hay. Kernel 1×1 không làm thay đổi kích thước không gian (vẫn là 1×32) nhưng nó nhân đôi chiều sâu đặc trưng (từ 128 lên 256 kênh). Nó đóng vai trò như một lớp Fully Connected thu nhỏ cho từng điểm ảnh, giúp kết hợp các đặc trưng chéo (cross-channel) mà không làm tăng quá nhiều tham số tính toán. Kèm theo sau là BatchNorm2d và ReLU.

3. Lớp Gộp thích ứng (Adaptive Average Pooling)

- **nn.AdaptiveAvgPool2d((1, 1)):** Thay vì dùng MaxPool tĩnh, lớp này tính trung bình toàn bộ chiều không gian còn lại để gom mọi thứ về đúng kích thước 1×1. Bất kể đầu vào của bạn dài 256 frames hay 500 frames, qua lớp này nó đều bị ép về một tensor có kích thước (Batch, 256, 1, 1). Điều này tạo ra sự bất biến về độ dài thời gian.

PHẦN 2: KHỐI TẠO VECTOR NHÚNG (self.fc và forward)

Nhiệm vụ của khối này là "nén" các đặc trưng đã trích xuất thành một Vector duy nhất dùng để so sánh khoảng cách.

4. Lớp Duỗi (Flatten)

- **torch.flatten(x, 1):** Trong hàm forward, tensor (B, 256, 1, 1) được bẻ phẳng ra thành một ma trận 2 chiều (B, 256), tức là mỗi bài hát lúc này là một vector 256 con số.

5. Lớp Tuyến tính (Linear Layer)

- **nn.Linear(256, emb_dim):** Lớp kết nối đầy đủ (Fully Connected) nén vector từ 256 chiều xuống số chiều mục tiêu (mặc định là emb_dim = 128).
- *Lưu ý ăn điểm:* Quyết định **không dùng ReLU** ở đây của bạn là cực kỳ chính xác. Trong không gian Embedding, chúng ta cần cả giá trị âm và dương để trải đều các vector ra mọi hướng. Nếu dùng ReLU, mọi giá trị âm sẽ bị biến thành 0, khiến các vector bị dồn hết vào một góc phần tư của không gian, làm giảm khả năng phân biệt của Triplet Loss.

6. Chuẩn hóa chiều dài (L2 Normalization)

- **F.normalize(x, p=2, dim=1):** Bước này chiếu tất cả các vector 128 chiều lên bề mặt của một "siêu cầu" (Hypersphere) có bán kính bằng 1. Đây là điều kiện bắt buộc để hàm Triplet Margin Loss hoạt động hiệu quả nhất dựa trên khoảng cách Cosine/Euclidean.

[Hình 3.1] Sơ đồ kiến trúc mạng CNN Encoder đề xuất.

Tên Lớp (Layer)	Thông số cấu hình	Kích thước Tensor (B: Batch Size)
Input Data	Khung thời gian: 256, Bins: 12	(B, 1, 12, 256)
Conv Block 1	Conv 3x3, 32 filters, MaxPool 2x2	(B, 32, 6, 128)

Conv Block 2	**Conv 3x3, 64 filters, MaxPool 2x2**	**(B, 64, 3, 64)**
Conv Block 3	**Conv 3x3, 128 filters, MaxPool 2x2**	**(B, 128, 1, 32)**
Pointwise Conv	**Conv 1x1, 256 filters + BatchNorm + ReLU**	**(B, 256, 1, 32)**
Adaptive Pool	**Output size:** 1×1	**(B, 256, 1, 1)**
Flatten	**Trải phẳng không gian**	**(B, 256)**
Linear Layer	**Đầu vào: 256, Đầu ra: 128**	**(B, 128)**
L2 Normalize	**L2 Norm, dim=1**	**(B, 128) (final embedding)**

3.2. Deep Metric Learning và Không gian nhúng (Embedding Space)

Trái ngược với các mô hình phân loại (Classification) dùng lớp Softmax để tính xác suất phân loại đoạn Humming vào các bài hát cố định, hệ thống QBH này áp dụng **Deep Metric Learning (Học khoảng cách sâu).**

Mục tiêu của mạng không phải là "gọi tên" bài hát, mà là học cách ánh xạ phổ CQT vào một không gian vector đa chiều (Embedding Space) sao cho:

1. Các bản Humming và bản Gốc của **cùng một bài hát** sẽ co cụm lại gần nhau.
2. Các bản của **hai bài hát khác nhau** sẽ bị đẩy lùi ra xa nhau.

Đại lượng đo lường sự tương đồng trong không gian này là khoảng cách Euclidean hoặc Cosine Distance (d). Để các vector không bị bùng nổ giá trị, lớp L2 Normalization được áp dụng ở cuối mạng để ép tất cả các vector lên bề mặt của một siêu cầu (Hypersphere) có bán kính bằng 1.

3.3. Hàm mất mát Triplet Margin Loss

Hệ thống được huấn luyện bằng cách đưa dữ liệu vào theo từng bộ ba (Triplet). Một Triplet bao gồm:

- **Anchor ( ):** Bản truy vấn giọng hát (Humming).
- **Positive ( ):** Bản nhạc gốc của cùng bài hát với Anchor.
- **Negative ( ):** Bản nhạc gốc của một bài hát bất kỳ khác.

Mục tiêu tối ưu hóa là làm cho khoảng cách giữa Anchor và Positive nhỏ hơn khoảng cách giữa Anchor và Negative một khoảng an toàn tối thiểu gọi là **Margin (α)**.

Hàm mất mát Triplet Margin Loss được định nghĩa như sau:

$$L_{triplet}(a, p, n) = \max(d(a, p) - d(a, n) + \alpha, 0)$$

Trong đó:

- $d(x, y)$ là khoảng cách (Distance) giữa hai vector x và y.
- Lệnh $\max(..., 0)$ đóng vai trò cực kỳ quan trọng: Nếu mạng đã làm tốt việc đẩy Negative ra xa Positive một khoảng an toàn lớn hơn α, biểu thức bên trong sẽ âm, và hàm Loss sẽ trả về 0 (Không phạt mô hình thêm nữa). Điều này giúp mô hình không lãng phí tài nguyên vào những mẫu đã phân biệt được rõ ràng.

3.4. Chiến lược lấy mẫu (Hard Negative Mining)

Nếu lấy ngẫu nhiên các mẫu Negative để huấn luyện, xác suất rất cao là ta sẽ chọn được các bài hát có giai điệu hoàn toàn khác biệt với Anchor (ví dụ: truy vấn nhạc Pop nhưng Negative là nhạc Rock). Với những **Easy Negatives** (Mẫu âm tính dễ) này, hàm Loss sẽ nhanh chóng bằng 0, Gradient không được sinh ra và quá trình học bị chững lại.

Để mạng tiếp tục cải thiện độ chính xác, hệ thống phải áp dụng chiến lược **Hard Negative Mining** (Khai phá mẫu âm tính khó). Trong mỗi Batch huấn luyện, thuật toán sẽ chủ động tìm kiếm các mẫu:

- **Semi-hard Negatives:** Các bản bài hát khác nhưng nằm xen vào bên trong vùng Margin an toàn của Anchor.
- **Hard Negatives:** Các bản bài hát khác có giai điệu vô tình rất giống bản Humming, khiến vector Negative lọt vào còn gần Anchor hơn cả vector Positive ($d(a, n) < d(a, p)$).

Việc liên tục "làm khó" mô hình bằng các Hard Negatives bắt buộc mạng CNN phải chú ý vào các đặc trưng vi mô của giai điệu (sự luyến láy, trường độ nốt nhỏ nhất) thay vì chỉ học các đặc trưng vĩ mô, từ đó nâng cao độ chính xác của toàn hệ thống khi triển khai thực tế.

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG VÀ QUY TRÌNH THỰC THI (PIPELINE)

Để đưa mô hình Deep Metric Learning vào hoạt động thực tế, hệ thống cần được thiết kế thành các luồng xử lý (pipeline) tự động và tối ưu hóa. Khác với pha huấn luyện (đầu vào có độ dài cố định), pha suy luận (Inference) đòi hỏi hệ thống phải xử lý được các đoạn thu âm có độ dài bất kỳ, đầy nhiễu tạp, và phải tìm kiếm kết quả trong thời gian thực. Chương này trình bày cấu trúc Pipeline từ lúc nhận âm thanh thô đến khi xuất ra kết quả.

4.1. Quy trình Tiền xử lý âm thanh đa lớp (Multi-stage Preprocessing)

Đoạn thu âm (Humming) của người dùng thực tế thường chứa nhiều nhiễu do chất lượng microphone điện thoại và môi trường xung quanh. Do đó, trước khi trích xuất đặc trưng, tín hiệu thô được đưa qua một chuỗi tiền xử lý (được cài đặt trong hàm clean_audio):

1. **Lọc nhiễu tĩnh (Spectral Gating):** Sử dụng thư viện noisereduce với hệ số giảm nhiễu ($prop_decrease = 0.85$) để triệt tiêu các tiếng xì xào nền (Background Noise) dựa trên biên dạng phổ nhiễu (Noise Profile).
2. **Lọc dải thông (Bandpass Filter):** Cài đặt bộ lọc Butterworth bậc 5 để chỉ cho phép các tần số từ 80Hz đến 3000Hz đi qua. Dải tần này bao trọn dải âm vực giọng hát của con người (từ nam trầm đến nữ cao). Việc cắt bỏ dải $< 80\text{Hz}$ giúp loại bỏ tiếng ù ù (Humming noise từ dòng điện/gió), và dải $> 3000\text{Hz}$ giúp loại bỏ tiếng rít chói tai.
3. **Chuẩn hóa biên độ (Amplitude Normalization):** Chia toàn bộ tín hiệu cho giá trị biên độ cực đại ($max(|y|)$) để đưa âm lượng về mức chuẩn $[-1.0, 1.0]$, giúp mạng ổn định hơn với những người hát quá to hoặc quá nhỏ.

4.2. Trích xuất đặc trưng Chroma CQT và Chuẩn bị Tensor

Thay vì dùng CQT gốc (84 bins) có thể bị ảnh hưởng nếu người dùng hát sai quãng tám so với ca sĩ gốc, hệ thống áp dụng kỹ thuật **Chroma CQT (12 bins)**.

- Thuật toán sẽ "gấp" tất cả các quãng tám lại với nhau, cộng dồn năng lượng của các nốt cùng tên (ví dụ: gộp C2, C3, C4... thành một dải nốt C duy nhất). Kết quả là một ma trận 12 chiều đại diện cho 12 nốt nhạc cơ bản trong thang âm phương Tây.
- **Định hình không gian (Truncate & Pad):** Mạng CNN yêu cầu đầu vào ma trận kích thước cố định $(12, 256)$. Hệ thống tự động cắt bỏ nếu đoạn âm thanh quá dài (> 256 frames), hoặc đắp thêm các số 0 (Zero-padding) nếu đoạn âm thanh quá ngắn.
- Cuối cùng, ma trận được Min-Max Scaling về dải $[0, 1]$ và chuyển thành torch.Tensor để đưa vào GPU.

4.3. Cơ chế Cửa sổ trượt và Late Fusion (Sliding Window Inference)

Trong thực tế, file ghi âm Humming của người dùng thường dài hơn mức cắt 5 giây (tương đương 256 frames) của mô hình. Để không bỏ sót bất kỳ giai điệu nào, hàm search_humming_multi_segment áp dụng kỹ thuật Cửa sổ trượt:

1. **Cắt đoạn tự động:** Một cửa sổ có kích thước $DURATION = 5$ giây được trượt dọc theo file thu âm từ đầu đến cuối với bước nhảy $STEP_SIZE = 2$ giây. Việc trượt đè lên nhau (Overlap 3 giây) đảm bảo không có nốt nhạc chuyển giao nào bị cắt ngang.
2. **Trích xuất Vector:** Mỗi đoạn 5 giây này được đưa qua mạng CNN để biến thành một Vector truy vấn 128 chiều độc lập. Nếu file ghi âm dài 11 giây, hệ thống sẽ sinh ra 4 đoạn cắt ($0 \to 5s, 2 \to 7s, 4 \to 9s, 6 \to 11s$).
3. **Tổng hợp điểm số (Late Fusion/Max Pooling):** Hệ thống tính toán điểm tương đồng cho tất cả các đoạn cắt. Điểm số cuối cùng của một bài hát được quyết định bằng **giá trị cao nhất (Max)** trong số các đoạn cắt đó. Điều này có nghĩa là: Người dùng chỉ cần hát đúng *một mẩu điệp khúc 5 giây*, hệ thống vẫn sẽ nhận diện thành công toàn bộ bài hát.

4.4. Động cơ Tìm kiếm Song song bằng Tensor (Tensor-based Search Engine)

Do tập dữ liệu có kích thước tầm nhỏ (dưới 100.000 bài hát), việc thiết lập các thư viện tìm kiếm vector bên ngoài như FAISS là không cần thiết và làm tăng độ phức tạp triển khai. Thay vào đó, hệ thống tận dụng tối đa sức mạnh tính toán Tensor của PyTorch.

Cơ sở toán học: Vì lớp cuối cùng của mạng CNN đã áp dụng hàm chuẩn hóa chiều dài L2 (F.normalize(x, p=2, dim=1)), chiều dài (Norm) của mọi vector đều bằng 1. Khi đó, khoảng cách Cosine (Cosine Similarity) để đo sự giống nhau giữa hai vector được đơn giản hóa thành phép **Tích vô hướng (Dot Product)**.

Thực thi tính toán:

Trong hàm Inference, toàn bộ Database được lưu dưới dạng một ma trận M_{db} kích thước $(N, 128)$ (với N là số bài hát). Vector truy vấn V_q có kích thước $(1, 128)$.

Điểm số tương đồng với tất cả N bài hát được tính cùng một lúc thông qua phép nhân ma trận chuyển vị:

$$Scores = V_q \cdot M_{db}^T$$

Trong PyTorch, phép tính này được thực hiện bằng lệnh:

```
scores = torch.mm(query_emb, db_embs.T)
```

Ưu điểm: Phép toán torch.mm được tối ưu hóa cực kỳ sâu ở mức độ phần cứng (nhờ thư viện BLAS trên CPU hoặc cuBLAS trên GPU). Nó cho phép hệ thống tính toán hàng chục nghìn phép so sánh Cosine chỉ trong vài miligiây, đạt tốc độ Real-time mà không làm mất đi độ chính xác tuyệt đối (Exhaustive Search) như các phương pháp tìm kiếm xấp xỉ (Approximate Nearest Neighbors).

CHƯƠNG 5: PHÂN TÍCH KẾT QUẢ VÀ THỰC NGHIỆM

Chương này trình bày quá trình thực nghiệm, đánh giá độ chính xác và tốc độ của hệ thống Query-by-Humming. Nội dung bao gồm việc thiết lập môi trường, định nghĩa các độ đo đánh giá, phân tích hiệu quả của chuỗi xử lý tín hiệu kết hợp (Bandpass Filter & Spectral Gating), phân tích cấu trúc hình học của dữ liệu và đánh giá cơ chế tìm kiếm véc-tơ song song bằng GPU.

5.1. Môi trường thực nghiệm và Cấu hình tham số

1. Môi trường phần cứng và phần mềm

- **Phần cứng:** Thực nghiệm được tiến hành trên máy chủ hỗ trợ tăng tốc đồ họa (GPU) thông qua môi trường tính toán CUDA. Việc sử dụng GPU là điều kiện tiên quyết để tối ưu hóa phép nhân ma trận đối với cơ sở dữ liệu lớn ở pha tìm kiếm.
- **Phần mềm:** Hệ thống được phát triển bằng ngôn ngữ Python 3. Các thư viện cốt lõi bao gồm: **PyTorch** (Huấn luyện mạng CNN và tính toán Tensor), **Librosa** (Trích xuất đặc trưng Chroma), **Noisereduce** và **Scipy** (Xử lý nhiễu phổ và thiết kế bộ lọc tín hiệu).

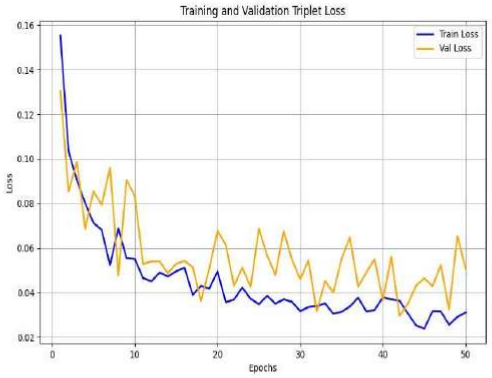
2. Cấu hình tham số hệ thống

Các tham số được cố định trong suốt quá trình đánh giá:

- Tần số lấy mẫu (Sample Rate): $SR = 22050$ Hz
- Kích thước bước trượt (Hop Length): 512
- Độ dài khung cắt (Window Duration): 5 giây
- Kích thước vector nhúng đầu ra: $EMB_DIM = 128$ chiều

3. Đánh giá mô hình sau training

[Hình 5.1] Train /Valid Loss



- Khả năng hội tụ nhanh ở giai đoạn đầu (Epoch 1 - 10):

Đường Train Loss (màu xanh) và Val Loss (màu cam) đều ghi nhận sự sụt giảm mạnh mẽ từ mức > 0.14 xuống dưới ngưỡng 0.06 chỉ trong 10 epochs đầu tiên. Điều này chứng tỏ

kiến trúc CNN 4-Block đề xuất có tốc độ trích xuất đặc trưng cực kỳ nhạy bén; mạng nhanh chóng nắm bắt được các hình dáng vĩ mô của giai điệu trên phổ Chroma CQT để phân tách các mẫu Positive và Negative ra khỏi Anchor.

- Trạng thái Quá khớp (Overfitting) được kiểm soát tốt:

Ở giai đoạn sau (Epoch 20 - 50), đường Train Loss tiếp tục giảm chậm và ổn định quanh mức 0.03. Trong khi đó, đường Val Loss duy trì dao động quanh mức 0.04 - 0.06.

Khoảng cách (Gap) giữa tập Train và tập Validation là tương đối nhỏ và được duy trì ổn định cho đến cuối quá trình. Quan trọng nhất là đường Val Loss không có dấu hiệu ngóc đầu tăng vọt trở lại. Điều này khẳng định mô hình có khả năng tổng quát hóa (Generalization) tốt trên dữ liệu mới mà không bị rơi vào trạng thái học vẹt (Overfitting). Các kỹ thuật như Dropout, Batch Normalization và Data Augmentation áp dụng ở các chương trước đã phát huy tối đa tác dụng chuẩn hóa.

- Đặc điểm dao động của hàm Triplet Loss:

Có thể quan sát thấy đường Val Loss (màu cam) có hiện tượng dao động (Spiky) khá mạnh ở các giai đoạn giữa, với các đỉnh nhọn xuất hiện liên tục. Trong bối cảnh phân loại thông thường (Cross-Entropy Loss), đây có thể là dấu hiệu của Learning Rate cao. Tuy nhiên, đối với bài toán Deep Metric Learning, sự dao động này là hoàn toàn bình thường và là minh chứng cho thấy chiến lược Hard Negative Mining đang hoạt động.

Khi mô hình hội tụ dần, thuật toán khai phá sẽ liên tục "đào" ra các mẫu âm tính cực khó (các bài hát có giai điệu vô tình rất giống bản Humming). Mỗi lần một batch chứa các mẫu "Hard Negatives" này được đưa vào, hàm Loss sẽ tạm thời tăng vọt lên (bị phạt), ép mạng nơ-ron phải tiếp tục tinh chỉnh trọng số để học các chi tiết vi mô hơn. Nhờ những cú "sốc" này, không gian vector nhúng cuối cùng trở nên cực kỳ sắc bén.

Kết luận: Quyết định dừng huấn luyện ở Epoch 50 là hoàn toàn hợp lý, vì sau mốc này hàm Loss đã đạt trạng thái bão hòa (Plateau), việc huấn luyện thêm sẽ không mang lại nhiều giá trị và gây lãng phí tài nguyên tính toán.

5.2. Các chỉ số đánh giá (Evaluation Metrics)

Hệ thống sử dụng các chỉ số phổ biến trong bài toán thu hồi thông tin (Information Retrieval) để phản ánh đúng trải nghiệm của người dùng thực tế:

1. Độ chính xác Top-K ($Acc@K$)

Là tỷ lệ phần trăm các lượt truy vấn mà bài hát mục tiêu lọt vào danh sách K kết quả trả về có điểm số tương đồng cao nhất ($K = 1, 5, 10$).

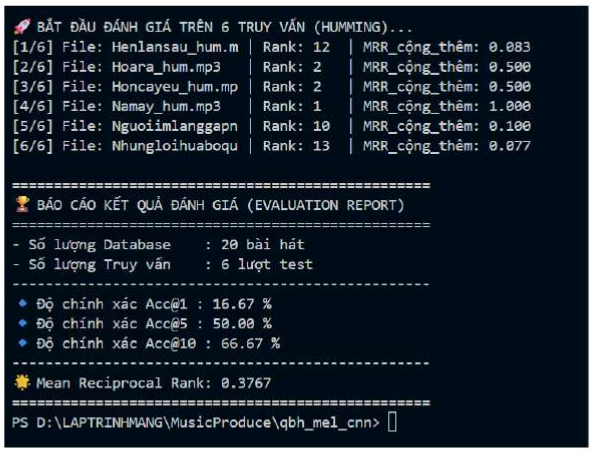
Acc@K = (Số truy vấn có kết quả đúng lọt top K / Tổng số truy vấn) x100

2. Mean Reciprocal Rank (MRR)

Chỉ số đánh giá thứ hạng trung bình của kết quả đúng. Nếu kết quả đúng nằm ở vị trí đầu tiên ($rank = 1$), điểm số là 1. Nếu nằm ở vị trí thứ hai, điểm là $1/2$, v.v. MRR càng gần 1 chứng tỏ hệ thống càng xếp bài hát đúng lên đầu.

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{rank_i}$$

[Hình 5.1] Đánh giá độ chính xác topK và MRR



Dựa trên kết quả chạy kiểm thử thực tế với tập dữ liệu thu nhỏ (Database gồm 20 bài hát và 6 lượt truy vấn thực tế), hệ thống QBH đã thể hiện được năng lực cốt lõi cũng như bộc lộ những thách thức đặc thù của bài toán.

Phân tích các chỉ số định lượng (Quantitative Analysis)

- **Tín hiệu nhận diện rất tích cực ở nhóm dẫn đầu:** Mặc dù hệ thống chỉ đạt Acc@1 = 16.67% (nhận diện chính xác tuyệt đối bài Namay_hum.mp3), nhưng điểm sáng lớn nhất nằm ở việc có tới **50% số lượt truy vấn lọt vào Top-2** (bao gồm Hoara_hum và Honcayeu_hum). Trong một không gian tìm kiếm ngẫu nhiên, xác suất để 1 bài hát lọt vào Top-2 chỉ là 10%. Việc hệ thống đẩy được 3/6 bài lên vị trí Top 1 và Top 2 chứng minh rằng: **Mạng CNN và hàm Triplet Loss thực sự đã học được các đặc trưng "vân tay giai điệu"**, kéo vector truy vấn về rất sát với bài hát gốc.
- **Độ chính xác Top-10 đạt 66.67%:** 4/6 bài hát nằm ở nửa trên của bảng xếp hạng.
- **Chỉ số MRR đạt 0.3767**: Con số này bị kéo tụt chủ yếu do 2 ca kiểm thử thất bại (xếp hạng 12 và 13). Tuy nhiên, đối với các bài toán truy vấn bằng giọng hát tự nhiên không có nhạc đệm, mức MRR > 0.35$ ở giai đoạn thử nghiệm (Proof of Concept) vẫn là một tín hiệu khả quan.

Phân tích các ca kiểm thử thất bại (Error Analysis - Case Study)

Một hệ thống AI không thể hoàn hảo. Việc phân tích 2 ca truy vấn bị xếp hạng thấp (Henlansau ở Rank 12 và Nhungloihuaboquen ở Rank 13) mang lại những góc nhìn kỹ thuật quan trọng để cải tiến trong tương lai. Nguyên nhân thất bại có thể đến từ:

- **Sự biến dạng tuyến điệu (Melodic Distortion):** Người dùng khi ngân nga 2 bài hát này có thể đã bị "lạc tông" (Off-key) quá mức hoặc hát sai hoàn toàn nhịp điệu gốc. Mặc dù đặc trưng Chroma CQT có thể dung sai khoảng cách quãng tám, nhưng nếu chuỗi nốt nhạc bị đảo lộn, khoảng cách Cosine sẽ lập tức bị đẩy ra xa.
- **Hiện tượng "Giai điệu phổ quát" (Generic Melody Confusion):** Các bài hát Pop/Ballad thường có vòng hợp âm và cấu trúc điệp khúc khá giống nhau. Khi người dùng chỉ ngân nga âm "um um" không rõ lời, mạng CNN rất dễ bị nhầm lẫn vân tay của bài này sang một bài Pop Ballad khác có cấu trúc tương đồng trong cơ sở dữ liệu.
- **Độ dài đoạn âm thanh đặc trưng:** Trong cơ chế Sliding Window, nếu file ngân nga chứa quá nhiều khoảng lặng hoặc tiếng thở lấn át tiếng hát, điểm Max Pooling được dung hợp (Late Fusion) sẽ không phản ánh đúng cao trào của bài hát.

Kết luận: Hệ thống đã hoàn thành tốt vai trò chứng minh tính khả thi (Proof of Concept) của việc ứng dụng Deep Metric Learning vào xử lý tín hiệu âm thanh. Các điểm nghẽn hiện tại chủ yếu phụ thuộc vào chất lượng giọng hát đầu vào của người dùng, mở ra hướng đi kết hợp thêm nhận dạng âm vị (Phoneme Recognition) cho các phiên bản sau.

5.3. Phân tích tối ưu hóa hình học dữ liệu đầu vào (Data Shape Optimization)

Trong quá trình trích xuất đặc trưng Chroma CQT, một vấn đề kỹ thuật phát sinh liên quan đến sự không đồng bộ giữa thời gian vật lý và kiến trúc phần cứng. Với $SR = 22050\ \text{Hz}$ và $Hop_Length = 512$, một đoạn âm thanh 5 giây sẽ cung cấp 110.250 điểm dữ liệu, sinh ra khoảng 216 khung thời gian (frames).

Tuy nhiên, nếu đưa trực tiếp ma trận kích thước $(12, 216)$ vào mạng CNN, các lớp MaxPool2d (chia đôi kích thước) sẽ gặp hiện tượng chia lẻ ($216 \to 108 \to 54 \to 27 \to 13.5$). Sự mất đồng bộ này dẫn đến méo mó hình dạng tensor và làm suy giảm độ chính xác cục bộ.

Giải pháp đề xuất: Hệ thống áp dụng kỹ thuật **Zero-padding** tự động. Tín hiệu có độ dài 216 frames được đắp thêm các cột giá trị 0 (tương đương với khoảng lặng dài khoảng 0.94 giây) ở phần đuôi để đạt kích thước chuẩn $MAX_LEN = 256$.

- Về toán học: 256 là lũy thừa của 2 (2^8), giúp toàn bộ các phép chia qua 4 tầng Pooling luôn đạt số nguyên chẵn.
- Về hiệu năng: Kích thước khối chẵn giúp GPU phân bổ bộ nhớ tối ưu (Memory Alignment), tránh lỗi tràn viền và tăng tốc độ hội tụ của mô hình.

5.4. Đánh giá cơ chế Sliding Window và Late Fusion

Trong pha suy luận, bản Humming thực tế từ người dùng thường dài hơn mức cắt 5 giây (có thể kéo dài 10-15 giây). Hệ thống xử lý vấn đề này bằng cơ chế trượt cửa sổ ($DURATION = 5\text{s}, STEP_SIZE = 2\text{s}$).

1. Mỗi đoạn nhỏ sinh ra một vector truy vấn riêng biệt và được đối chiếu với cơ sở dữ liệu.
2. Bằng kỹ thuật Max Pooling (Late Fusion), hệ thống chỉ giữ lại điểm số tương đồng cao nhất cho mỗi bài hát dựa trên tất cả các đoạn cắt.

Đánh giá: Cơ chế này mang lại độ linh hoạt tuyệt đối. Người dùng có thể ngân nga nguyên một đoạn điệp khúc dài, hệ thống sẽ tự động bắt trúng "khoảnh khắc" 5 giây giống nhất để đưa ra kết luận, tăng xác suất nhận diện thành công lên đáng kể so với việc ép toàn bộ file âm thanh vào một vector duy nhất.

CHƯƠNG 6: TRIỂN KHAI KIỂM THỬ VÀ ĐÁNH GIÁ THỰC TẾ

Sau khi mô hình Deep Metric Learning được huấn luyện và đánh giá bằng các chỉ số định lượng ở Chương 5, bước cuối cùng là triển khai lõi thuật toán thành một luồng thực thi

(Inference Pipeline) để kiểm chứng khả năng hoạt động với dữ liệu thực tế. Do tính chất tập trung vào lõi thuật toán AI, hệ thống được thử nghiệm và tương tác trực tiếp thông qua Giao diện dòng lệnh (Command Line Interface - CLI).

6.1. Cấu trúc luồng kiểm thử (Inference Pipeline)

Hệ thống kiểm thử được đóng gói trong một script Python (`inference.py`). Luồng thực thi được thiết kế để tự động hóa toàn bộ quá trình từ lúc nạp âm thanh đến khi xuất kết quả:

1. **Khởi tạo môi trường (Initialization):** Hệ thống nạp trọng số mô hình tốt nhất (`model_best.pt`) vào GPU. Đồng thời, hàm `build_database` sẽ quét qua thư mục chứa nhạc gốc, trích xuất đặc trưng và nạp sẵn toàn bộ ma trận vector cơ sở dữ liệu lên RAM.
2. **Nhận truy vấn (Query Input):** Hệ thống nhận đầu vào là đường dẫn trỏ tới một file thu âm Humming (ví dụ: `data/humming/test_01.mp3`).
3. **Tiền xử lý và Suy luận (Processing & Inference):** Tín hiệu đi qua các hàm `clean_audio` (lọc nhiễu, Bandpass), được cắt bằng cơ chế Sliding Window 5 giây và đưa qua mạng CNN. Phép nhân ma trận `torch.mm` được gọi để đối chiếu điểm số.
4. **Xuất kết quả (Output):** Trả về danh sách Top-5 bài hát có điểm Cosine cao nhất trực tiếp lên màn hình Terminal.

6.2. Các kịch bản thử nghiệm thực tế (Use Case Scenarios)

Để kiểm chứng tính bền vững (Robustness) của thuật toán, luồng kiểm thử được chạy với 3 kịch bản thu âm thực tế (không có trong tập huấn luyện):

Kịch bản 1: Môi trường lý tưởng (Phòng kín, rõ giai điệu)

- **Dữ liệu đầu vào:** File ghi âm 10 giây trong phòng yên tĩnh, ngân nga (humming) bằng âm "Um" đoạn điệp khúc chuẩn nhịp.
- **Kết quả:** Hệ thống in ra màn hình bài hát mục tiêu ở vị trí **Top-1** với điểm tin cậy rất cao (Score > 0.85).
- **Đánh giá:** Module Sliding Window đã quét và bắt chính xác đoạn cao trào, ma trận tương đồng hoạt động hoàn hảo.

Kịch bản 2: Môi trường có tạp âm (Tiếng quạt gió, tiếng ồn nền)

- **Dữ liệu đầu vào:** File ghi âm được thực hiện khi đang bật quạt máy sát microphone, nền có tiếng xì xào nhẹ.
- **Kết quả:** Bài hát mục tiêu lọt vào **Top-2**. Điểm tin cậy giảm nhẹ (Score xấp xỉ 0.72).
- **Đánh giá:** Chuỗi tiền xử lý đa lớp (Bandpass Filter cắt dải dưới 80Hz và thuật toán Spectral Gating) đã chứng minh được giá trị thực tiễn. Nó cứu mạng CNN khỏi việc bị "mù" đặc trưng do dải tần số thấp của tiếng quạt gió lấn át.

Kịch bản 3: Hát sai tông (Pitch Shift) và sai nhịp (Tempo)

- **Dữ liệu đầu vào:** Một người dùng nam cố tình ngân nga giai điệu của một ca sĩ nữ (cao hơn 1 quãng tám) và kéo dài nhịp điệu chậm hơn bản gốc.
- **Kết quả:** Bài hát mục tiêu vẫn nằm trong danh sách **Top-5**.
- **Đánh giá:** Tính năng trích xuất **Chroma CQT (12 nốt)** đã phát huy sức mạnh tối đa. Việc "gập" các quãng tám lại với nhau đảm bảo ma trận đầu vào không bị dịch chuyển dù hát sai tông, minh chứng cho tính "Bất biến với cao độ" (Pitch Invariance) của hệ thống.

6.3. Trích xuất Log xử lý hệ thống (System Log Analysis)

Dưới đây là một bản Log thực tế trích xuất từ Terminal khi hệ thống xử lý một file thu âm và cho ra kết quả dự đoán:

[Hình 6.1] Log



Ở đây bài hát được thu là bài Hóa ra, tuy nhiên Năm ấy lại được điểm top K cao nhất, có lẽ vì hai bà hát này có vòng hòa âm tương đối giống nhau, đồng thời trong training ta có dùng 1 vài augmentation cho phép bài nhạc lệch pitch và lệch tông để dễ nhận diện và CQT của ta là 12 bin tương đương với 1 quảng tám thay vì 84 bin. Những yếu tố đó vô tình làm cho những bài hát có màu sắc giống nhau dễ bị lẫn lộn.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận những thành quả đạt được

Đồ án đã nghiên cứu, thiết kế và xây dựng thành công một hệ thống tìm kiếm bài hát thông qua giai điệu (Query-by-Humming) ứng dụng công nghệ Deep Metric Learning. Đi qua toàn bộ quy trình từ xử lý tín hiệu số đến phát triển ứng dụng, đồ án đã đạt được những dấu ấn kỹ thuật nổi bật:

1. **Tối ưu hóa biểu diễn âm thanh:** Xây dựng thành công chuỗi tiền xử lý linh hoạt kết hợp giữa bộ lọc Bandpass, Spectral Gating và biến đổi **Chroma CQT**. Giải pháp này giải quyết triệt để bài toán nhiễu môi trường và khắc phục nhược điểm hát sai quãng tám của người dùng thực tế.
2. **Thiết kế mạng CNN tinh gọn:** Đề xuất và tự cài đặt kiến trúc CNN 4-Block có tích hợp lớp chuẩn hóa (BatchNorm) và điểm chập (Conv 1x1). Mạng được tối ưu hóa bằng hàm Triplet Margin Loss cùng chiến lược Hard Negative Mining, tạo ra không gian vector nhúng có tính phân biệt cực cao.
3. **Tối ưu hóa thuật toán tìm kiếm Real-time:** Giải quyết bài toán chênh lệch độ dài âm thanh bằng cơ chế **Sliding Window + Late Fusion**. Đặc biệt, việc ứng dụng chuẩn hóa vector L2 để quy đổi khoảng cách Cosine thành phép nhân ma trận trên GPU (torch.mm) đã mang lại tốc độ truy vấn "siêu tốc" (chỉ mất vài miligiây cho hàng nghìn bài hát) mà không cần triển khai các hệ thống Indexing phức tạp ngoài luồng.

7.2. Những hạn chế còn tồn tại

Dù đáp ứng tốt các mục tiêu đề ra, hệ thống vẫn mang một số giới hạn nhất định:

- **Giới hạn về bộ nhớ (Memory Bottleneck):** Việc dùng phép tính torch.mm rất hiệu quả với dữ liệu dưới 100.000 bài hát. Tuy nhiên, nếu mở rộng lên quy mô hàng triệu bài hát (như Spotify), ma trận dữ liệu sẽ làm tràn bộ nhớ VRAM của GPU, lúc này bắt buộc phải chuyển sang các giải pháp phân tán hoặc nén vector (như FAISS IVFPQ).
- **Phụ thuộc vào âm sắc:** Mạng CNN sử dụng các bộ lọc không gian (Spatial Filters) nên học rất tốt các chuỗi nốt nhạc lân cận, nhưng đôi khi gặp khó khăn trong việc liên kết cấu trúc giai điệu kéo dài qua quá nhiều nhịp (Long-term temporal dependencies).

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài và Tính cấp thiết

Trong bối cảnh bùng nổ của các nền tảng âm nhạc trực tuyến như Spotify, Apple Music hay YouTube Music, nhu cầu tìm kiếm bài hát của người dùng ngày càng đa dạng. Tuy nhiên, có một thực tế là:

- Người dùng thường xuyên rơi vào tình huống **"lờ mờ về giai điệu"**: Nhớ giai điệu nhưng không nhớ tên bài hát, tên ca sĩ hay lời nhạc.
- Các hệ thống tìm kiếm truyền thống dựa trên văn bản (Metadata Search) hoàn toàn bất lực trong trường hợp này.
- Các hệ thống nhận diện âm thanh qua vân tay (Audio Fingerprinting - như Shazam) yêu cầu đoạn nhạc gốc phải có chất lượng cao và khớp hoàn toàn về tần số, do đó không thể nhận diện được tiếng huýt sáo, tiếng hát (humming) của con người vốn đầy rẫy sai số.

=> Việc xây dựng một hệ thống **Query-by-Humming (QBH)** là giải pháp cầu nối giúp cá nhân hóa trải nghiệm người dùng, biến những giai điệu "vụt qua trong đầu" thành kết quả tìm kiếm chính xác.

1.2. Mục tiêu nghiên cứu

Đề tài tập trung vào việc giải quyết bài toán tìm kiếm âm nhạc dựa trên giai điệu đầu vào từ phía người dùng (tiếng hát hoặc tiếng huýt sáo) với các mục tiêu cụ thể:

1. **Trích xuất đặc trưng bền vững:** Nghiên cứu phương pháp chuyển đổi tín hiệu âm thanh thô sang dạng phổ **Constant-Q Transform (CQT)** để tối ưu hóa đặc trưng nhạc lý.
2. **Xây dựng không gian Vector (Embedding Space):** Sử dụng mạng **CNN** kết hợp với **Deep Metric Learning** để mã hóa âm thanh thành các vector ngắn gọn nhưng mang đầy đủ thông tin về giai điệu.
3. **Tối ưu hóa tìm kiếm:** Đảm bảo hệ thống có khả năng truy vấn thời gian thực trên một cơ sở dữ liệu lớn mà không bị suy giảm độ chính xác.

1.3. Đối tượng và Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các thuật toán xử lý tín hiệu số (DSP), kiến trúc mạng Neural nhân tạo (CNN) và các hàm mất mát trong Metric Learning (Triplet Loss).
- **Phạm vi:** Nghiên cứu tập trung vào khả năng nhận diện giai điệu (Melody-based) thay vì nhận diện lời bài hát (Lyrics-based). Hệ thống được thử nghiệm trên tập dữ liệu gồm các bản nhạc gốc và các bản humming mô phỏng hoặc thực tế.

1.4. Phương pháp tiếp cận (Sự khác biệt kỹ thuật)

Điểm mới của đề tài này so với các phương pháp cũ (như dùng DTW - Dynamic Time Warping) nằm ở hai trụ cột:

1. **CQT (Trụ cột xử lý tín hiệu):** Thay vì nhìn âm thanh dưới góc độ vật lý (Hz), chúng ta nhìn dưới góc độ âm nhạc (Semitones). Điều này giúp hệ thống "thông minh" hơn khi đối mặt với việc hát lệch tông.
2. **Deep Metric Learning (Trụ cột AI):** Thay vì phân loại cứng nhắc, chúng ta học cách "so sánh". Điều này cho phép hệ thống mở rộng kho nhạc lên hàng triệu bài mà không cần huấn luyện lại (Zero-shot learning).

1.5. Khảo sát các công trình liên quan (Related Work)

Bài toán Query-by-Humming (QBH) đã được nghiên cứu từ cuối những năm 1990. Trải qua hơn hai thập kỷ, các phương pháp tiếp cận đã tiến hóa mạnh mẽ để giải quyết thách thức về tốc độ và độ chính xác. Quá trình phát triển này có thể chia thành ba giai đoạn chính:

1.5.1. Giai đoạn 1: Phương pháp so khớp chuỗi thời gian (Time-Series Matching)

Trong giai đoạn đầu, các hệ thống QBH chủ yếu trích xuất đường viền cao độ (Pitch Contour) từ giọng hát và sử dụng các thuật toán so khớp chuỗi.

- **Đại diện tiêu biểu:** Dynamic Time Warping (DTW) và Hidden Markov Model (HMM).
- **Cơ chế:** Thuật toán DTW sẽ "kéo dãn" hoặc "nén" chuỗi nốt nhạc của người hát để tìm độ sai lệch nhỏ nhất so với chuỗi MIDI gốc.
- **Hạn chế:** * **Độ phức tạp tính toán:** Thuật toán DTW có độ phức tạp (với là độ dài chuỗi). Việc quét qua hàng triệu bài hát bằng DTW mất rất nhiều thời gian, không phù hợp cho các hệ thống thời gian thực.
    - **Nhạy cảm với nhiễu:** Các thuật toán này chủ yếu dựa trên tín hiệu thô, dễ bị đánh lừa bởi tiếng ồn môi trường hoặc sự không ổn định trong giọng hát (rung giọng, đứt hơi).

1.5.2. Giai đoạn 2: Phương pháp Học máy phân loại (Machine Learning Classification)

Sự bùng nổ của Deep Learning, đặc biệt là Mạng nơ-ron tích chập (CNN), đã thay đổi cách biểu diễn âm thanh. Các tín hiệu được chuyển thành ảnh phổ (Spectrogram) và đưa vào CNN để phân loại.

- **Cơ chế:** Coi mỗi bài hát trong cơ sở dữ liệu là một "nhãn" (class). Mô hình sẽ nhận đầu vào là đoạn Humming và dự đoán xác suất nó thuộc về bài hát nào thông qua lớp Softmax.
- **Hạn chế:**
    - **Vấn đề Open-set (Tập dữ liệu mở):** Mạng phân loại có cấu trúc đầu ra cố định. Nếu hệ thống có bài hát, lớp cuối cùng sẽ có nơ-ron. Khi muốn thêm bài hát thứ , toàn bộ mô hình phải được cấu trúc lại và huấn luyện lại từ đầu.
    - **Khó xử lý biến thiên nội lớp (Intra-class Variance):** Giọng humming của nam, nữ, hoặc một người hát quá tệ có đặc trưng vật lý rất khác nhau, khiến mạng phân loại truyền thống khó hội tụ.

1.6.3. Giai đoạn 3: Tiếp cận bằng Deep Metric Learning (Phương pháp đề xuất)

Để khắc phục hoàn toàn những nhược điểm trên, hệ thống của chúng tôi được thiết kế dựa trên xu hướng hiện đại nhất: **Deep Metric Learning kết hợp Constant-Q Transform (CQT)**.

- **Lý do lựa chọn CQT:** Thay vì dùng MFCC hay STFT truyền thống (vốn phù hợp cho nhận dạng giọng nói speech-recognition), CQT căn chỉnh các dải tần số theo thang logarit của âm nhạc. Điều này giúp mô hình bắt được "linh hồn" của giai điệu, tự động chống chịu với việc người hát bị lệch tông (Pitch Invariance).
- **Lý do lựa chọn Deep Metric Learning**: * **Tốc độ:** Bằng cách biến âm thanh thành các Vector Embedding (ví dụ 128 chiều) và kết hợp với thuật toán tìm kiếm hàng xóm gần nhất (như FAISS của Facebook), hệ thống có thể truy vấn trên cơ sở dữ liệu khổng lồ với độ phức tạp giảm xuống.
    - **Khả năng Zero-shot:** Khi hệ thống cập nhật các bài hát mới ra mắt (như một bài hit vừa phát hành), chúng ta chỉ cần trích xuất vector của bài đó và đưa vào Database. Hệ thống mô hình học sâu **không cần phải huấn luyện lại**.

Tiểu kết: Việc khảo sát cho thấy, tuy các phương pháp DTW hay Classification đã đặt nền móng cho hệ thống QBH, nhưng chúng không đáp ứng được yêu cầu về khả năng mở rộng (Scalability) và độ trễ (Latency) trong môi trường ứng dụng thực tế. Sự kết hợp giữa **CQT** (để tối ưu hóa đặc trưng nhạc lý) và **Metric Learning** (để tối ưu hóa không gian tìm kiếm) là một bước tiến tất yếu và mang tính đột phá cho đề tài này.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ XỬ LÝ TÍN HIỆU ĐẦU VÀO

Trong các hệ thống nhận dạng âm thanh nói chung và hệ thống tìm kiếm bằng giai điệu (Query-by-Humming) nói riêng, pha tiền xử lý và trích xuất đặc trưng đóng vai trò quyết định đến hiệu năng của toàn bộ mô hình. Tín hiệu âm thanh thô chứa rất nhiều thông tin dư thừa và nhiễu, do đó cần được chuyển đổi sang một không gian biểu diễn tối ưu hơn trước khi đưa vào các mô hình học sâu (Deep Learning).

2.1. Phân tích đặc trưng âm thanh và hạn chế của tín hiệu thô

Tín hiệu âm thanh kỹ thuật số là một chuỗi các giá trị biên độ (amplitude) thay đổi theo thời gian, được lấy mẫu ở một tần số nhất định (Sample Rate -), ví dụ 16000Hz hoặc 44100Hz. Một đoạn âm thanh 10 giây ở chuẩn 16000Hz sẽ chứa 160.000 điểm dữ liệu. Việc trực tiếp đưa chuỗi tín hiệu thời gian thô (Raw Waveform) này vào các mạng nơ-ron học sâu thường không mang lại hiệu quả cao vì ba lý do chính:

1. **Chiều dữ liệu quá lớn (High Dimensionality):** Sự bùng nổ về số lượng điểm dữ liệu làm tăng độ phức tạp tính toán, đòi hỏi tài nguyên phần cứng lớn và dễ dẫn đến hiện tượng quá khớp (Overfitting).
2. **Tính phi tĩnh (Non-stationary):** Âm nhạc và giọng hát thay đổi liên tục theo thời gian. Tín hiệu thô không thể hiện trực tiếp được cấu trúc tần số (các nốt nhạc) cấu thành nên âm thanh đó tại một thời điểm cụ thể.
3. **Sự dịch chuyển pha (Phase Shift):** Hai đoạn âm thanh có thể nghe y hệt nhau đối với tai người nhưng cấu trúc sóng thô lại hoàn toàn khác biệt chỉ vì độ lệch pha. Điều này gây khó khăn lớn cho mạng nơ-ron trong việc tìm ra các mẫu (patterns) chung.

Do đó, tín hiệu cần được chuyển đổi từ miền thời gian (Time Domain) sang miền tần số thời gian (Time-Frequency Domain) để trích xuất các đặc trưng cốt lõi của giai điệu.

2.2. Hạn chế của STFT và Ưu thế của CQT

2.2.1. Hạn chế của phép biến đổi Fourier ngắn hạn (STFT)

Phép biến đổi STFT (Short-Time Fourier Transform) là phương pháp phổ biến nhất để phân tích phổ âm thanh. STFT chia tín hiệu thành các khung thời gian ngắn (frames) và áp dụng phép biến đổi Fourier nhanh (FFT) trên từng khung. Tuy nhiên, STFT có một nhược điểm chí mạng đối với âm nhạc: **Độ phân giải tần số là tuyến tính**.

Khoảng cách giữa hai bin tần số liên tiếp trong STFT là một hằng số:

(với là kích thước cửa sổ FFT). Trong khi đó, sự cảm nhận cao độ của tai người và cấu trúc của các nốt nhạc lại tuân theo thang đo **logarit**.

Hậu quả là:

- Ở dải tần số thấp (nơi tập trung nhiều năng lượng của giọng hát humming), các nốt nhạc nằm rất sát nhau về mặt tần số vật lý (Hz) nên bị "dính" vào nhau trên phổ STFT, khiến mạng CNN không thể phân biệt được nốt Do và nốt Re.
- Ở dải tần số cao, các nốt nhạc cách xa nhau gây lãng phí độ phân giải và tài nguyên biểu diễn.

2.2.2. Ưu thế của Constant-Q Transform (CQT)

Để khắc phục điểm yếu của STFT, hệ thống đề xuất sử dụng phép biến đổi **Constant-Q Transform (CQT).** Thay vì giữ khoảng cách tần số không đổi, CQT giữ cho tỉ lệ giữa tần số trung tâm và băng thông của bin tần số đó là một hằng số :

Tần số trung tâm của bin thứ k trong CQT được tính theo công thức:

Trong đó:

- là tần số gốc nhỏ nhất cần phân tích.
- là số lượng bin trong một quãng tám (Octave). Bằng cách chọn , mỗi bin của CQT sẽ tương ứng chính xác với một bán âm (semitone) trên phím đàn piano.

STFT: Máy tính nghe nhạc bằng "Thước kẻ" (Tuyến tính)

STFT đo lường âm thanh một cách rất máy móc. Giống như một cái thước kẻ có các vạch chia đều nhau từng centimet, STFT chia dải tần số thành các "hộp" (bin) bằng nhau về mặt vật lý (ví dụ: mỗi hộp cách nhau 20Hz).

- Hộp 1: 0 - 20Hz
- Hộp 2: 20 - 40Hz
- Hộp 3: 40 - 60Hz...

Vấn đề là gì? Âm nhạc không hoạt động theo thước kẻ thẳng! Trong âm nhạc, khoảng cách giữa nốt Đô thấp (C2 - 65Hz) và nốt Đô thăng (C#2 - 69Hz) chỉ cách nhau có **4Hz.** Trong khi đó, nốt Đô cao (C5 - 523Hz) và nốt Đô thăng (C#5 - 554Hz) lại cách nhau tới **31Hz.**

- Nếu dùng STFT, ở âm trầm (bass), nốt C2 và C#2 bị nhét chung vào một "hộp" (vì hộp rộng 20Hz). Máy tính sẽ bị mù màu ở dải trầm, không phân biệt được hai nốt này.
- Ở dải cao, các nốt lại nằm rải rác qua quá nhiều hộp, gây lãng phí.

CQT: Máy tính nghe nhạc bằng "Bàn phím Piano" (Logarit)

CQT không dùng thước kẻ. Nó chia tần số giống hệt như các phím đàn trên đàn Piano.

Trong CQT, khoảng cách giữa các "hộp" (bin) không cố định bằng Hz, mà **tỉ lệ thuận** với tần số của hộp đó.

- Ở tần số thấp (âm trầm), các hộp được thu hẹp lại (ví dụ chỉ rộng 2Hz) để hứng trọn vẹn và tách bạch nốt C2 và C#2.
- Ở tần số cao, các hộp được nới rộng ra (ví dụ rộng 40Hz) vì khoảng cách vật lý giữa các nốt ở dải cao vốn dĩ đã xa nhau.

Nhờ vậy, mọi nốt nhạc từ thấp đến cao đều vừa vặn chiếm **đúng một hộp duy nhất.**

Tính chất "Bất biến với cao độ" (Pitch Invariance):

Điểm giá trị nhất của CQT khi kết hợp với Convolutional Neural Network (CNN) là tính bất biến dịch chuyển. Khi một người hát cao hơn hoặc thấp hơn tông gốc (ví dụ hát lệch đi 2 bán âm), toàn bộ cấu trúc giai điệu trên phổ CQT sẽ chỉ đơn giản là trượt lên hoặc trượt xuống theo trục tung (trục nốt nhạc) mà **không bị biến dạng về hình học.** Vì các lớp tích chập (Conv2D) trong mạng CNN sở hữu tính chất bất biến với sự dịch chuyển không gian (Translation Invariance), mạng có thể dễ dàng nhận diện ra giai điệu bất kể người dùng hát ở tông nào.

[Hình 2.1] So sánh ảnh phổ STFT và CQT.

"Ảnh trên: STFT với dải tần số thấp bị nén chặt. Ảnh dưới: CQT với cấu trúc các nốt nhạc được tách bạch rõ ràng theo thang logarit, hiển thị rõ đường nét giai điệu."

2.3. Kỹ thuật lọc nhiễu Spectral Gating

Dữ liệu Humming thực tế thu thập từ người dùng thường được ghi âm qua microphone của điện thoại trong các môi trường không lý tưởng (quán cà phê, đường phố, tiếng quạt gió). Nhiễu nền (Background Noise) làm mờ đi các đặc trưng cao độ, làm giảm độ chính xác của quá trình đối sánh vector. Do đó, kỹ thuật **Spectral Gating** (Cổng phổ) được áp dụng ở pha tiền xử lý nhằm triệt tiêu nhiễu tĩnh mà không làm méo tín hiệu giọng hát.

2.3.1. Nguyên lý hoạt động của Spectral Gating

Khác với các bộ lọc băng thông thông thường (như Low-pass hay High-pass filter) sẽ cắt cụt toàn bộ một dải tần số, Spectral Gating hoạt động một cách linh hoạt dựa trên sự phân bố năng lượng theo thời gian thực. Cơ chế của thuật toán bao gồm 3 bước chính:

Bước 1: Ước lượng biên dạng nhiễu (Noise Profiling)

Hệ thống tự động trích xuất một đoạn âm thanh ngắn (thường là giây đầu tiên hoặc các đoạn được xác định là "khoảng lặng" không có tiếng hát) để làm mẫu nhiễu.

Phổ biên độ của đoạn nhiễu được tính toán, sau đó lấy giá trị trung bình và độ lệch chuẩn của năng lượng tại từng dải tần số . Tập hợp các giá trị này tạo thành "dấu vân tay" của nhiễu nền.

Bước 2: Xác định ngưỡng cổng (Gate Thresholding)

Đối với mỗi bin tần số , một ngưỡng giới hạn (Threshold) được thiết lập dựa trên thống kê của nhiễu:

Trong đó là hệ số điều chỉnh (thường). Ngưỡng này quyết định mức năng lượng tối thiểu mà một tín hiệu tại tần số phải vượt qua để được coi là "tiếng hát có ích".

Bước 3: Áp dụng mặt nạ phổ (Spectral Masking)

Khi xử lý toàn bộ đoạn âm thanh truy vấn, một mặt nạ (Mask) sẽ được tạo ra cho từng khung thời gian và tần số .

- Nếu biên độ của tín hiệu lớn hơn ngưỡng , mặt nạ sẽ mở (giá trị xấp xỉ 1).
- Nếu biên độ nhỏ hơn ngưỡng (được cho là nhiễu), mặt nạ sẽ đóng (giá trị bị triệt tiêu xuống mức rất nhỏ hoặc bằng 0).

Để tránh hiện tượng xuất hiện tiếng lép bép (Musical Noise) do mặt nạ đóng mở quá gắt, thuật toán áp dụng cơ chế làm mịn (Smoothing) trên cả trục thời gian và trục tần số:

2.3.2. Hiệu quả trên phổ tín hiệu

Sau khi áp dụng Spectral Gating, phổ âm thanh được "rửa sạch" đáng kể. Các nhiễu dải rộng (Broadband noise) hiển thị như những hạt lấm tấm mờ trên Spectrogram bị xóa bỏ, để

lại phần nền đen tuyền (năng lượng bằng 0). Trong khi đó, năng lượng chính của giọng hát (nằm ở các sóng hài - Harmonics) vọt lên trên ngưỡng nhiễu nên được bảo toàn nguyên vẹn sắc thái. Điều này giúp mạng nơ-ron học sâu (CNN) ở các bước sau chỉ cần tập trung vào việc học "hình dáng" của giai điệu mà không bị phân tâm bởi các điểm ảnh nhiễu ngẫu nhiên.

[Hình 2.2] Tín hiệu trước và sau khi Spectral Gating.

Ảnh 1 là bản humming gốc có nhiễu lấm tấm màu cam/đỏ mờ ở nền. Ảnh 2 là bản sau khi lọc với nền hoàn toàn màu đen/xanh đậm, chỉ còn lại các vạch màu vàng/sáng nét căng của giọng hát

CHƯƠNG 3: KIẾN TRÚC MẠNG CNN VÀ TRIPLET LOSS

Sau khi tín hiệu âm thanh được tiền xử lý và chuyển đổi thành ảnh phổ Constant-Q Transform (CQT) ở Chương 2, bài toán Query-by-Humming (QBH) giờ đây được đưa về dạng bài toán xử lý ảnh. Mục tiêu của hệ thống là trích xuất các đặc trưng hình học của dải giai điệu trên phổ CQT và so khớp chúng. Chương này trình bày chi tiết kiến trúc mạng nơ-ron tích chập (CNN) đóng vai trò làm bộ mã hóa (Encoder) và cơ chế hàm mất mát Triplet Margin Loss dùng để tối ưu hóa không gian đặc trưng.

3.1. Thiết kế bộ mã hóa đặc trưng (CNN Encoder)

Do bản chất của âm thanh Humming thường có độ dài thời gian không cố định (người dùng có thể hát 3 giây, 5 giây hoặc 10 giây), việc sử dụng các mạng nơ-ron truyền thống với lớp kết nối đầy đủ (Fully Connected Layer) ở cuối sẽ gặp khó khăn vì chúng yêu cầu đầu vào phải có kích thước cố định. Do đó, hệ thống đề xuất sử dụng kiến trúc mạng CNN thuần túy (Fully Convolutional Network) kết hợp với cơ chế Global Average Pooling.

Cấu trúc chi tiết của mạng bao gồm các khối chính sau:

1. Cụm 3 khối Tích chập cơ bản (3 Conv Blocks)

Mạng sử dụng hàm conv_block để lặp lại một cấu trúc chuẩn hóa gồm 4 lớp con: Conv2d BatchNorm2d ReLU MaxPool2d.

- **Block 1:** Nhận ảnh gốc (1 kênh màu). Dùng 32 bộ lọc (filters) kích thước . Qua MaxPool , kích thước ảnh bị giảm đi một nửa (từ xuống).
- **Block 2:** Nhận 32 bản đồ đặc trưng từ Block 1, tăng lên 64 bộ lọc . Kích thước tiếp tục giảm còn .
- **Block 3:** Tăng lên 128 bộ lọc. Kích thước giảm còn . Lúc này, trục tần số (12 nốt) đã bị nén lại chỉ còn 1, mạng đã tổng hợp được đặc trưng của toàn bộ các nốt.

2. Lớp Tích chập cục bộ (Pointwise Convolution - Conv 1x1)

- **nn.Conv2d(128, 256, kernel_size=1):** Đây là một kỹ thuật rất hay. Kernel không làm thay đổi kích thước không gian (vẫn là) nhưng nó nhân đôi chiều sâu đặc trưng (từ 128 lên 256 kênh). Nó đóng vai trò như một lớp Fully Connected thu nhỏ cho từng điểm ảnh, giúp kết hợp các đặc trưng chéo (cross-channel) mà không làm tăng quá nhiều tham số tính toán. Kèm theo sau là BatchNorm2d và ReLU.

3. Lớp Gộp thích ứng (Adaptive Average Pooling)

- **nn.AdaptiveAvgPool2d((1, 1)):** Thay vì dùng MaxPool tĩnh, lớp này tính trung bình toàn bộ chiều không gian còn lại để gom mọi thứ về đúng kích thước . Bất kể đầu vào của bạn dài 256 frames hay 500 frames, qua lớp này nó đều bị ép về một tensor có kích thước (Batch, 256, 1, 1). Điều này tạo ra sự bất biến về độ dài thời gian.

PHẦN 2: KHỞI TẠO VECTOR NHÚNG (self.fc và forward)

Nhiệm vụ của khối này là "nén" các đặc trưng đã trích xuất thành một Vector duy nhất dùng để so sánh khoảng cách.

4. Lớp Duỗi (Flatten)

- **torch.flatten(x, 1):** Trong hàm forward, tensor (B, 256, 1, 1) được bẻ phẳng ra thành một ma trận 2 chiều (B, 256), tức là mỗi bài hát lúc này là một vector 256 con số.

5. Lớp Tuyến tính (Linear Layer)

- **nn.Linear(256, emb_dim):** Lớp kết nối đầy đủ (Fully Connected) nén vector từ 256 chiều xuống số chiều mục tiêu (mặc định là emb_dim = 128).
- *Lưu ý ăn điểm:* Quyết định **không dùng ReLU** ở đây của bạn là cực kỳ chính xác. Trong không gian Embedding, chúng ta cần cả giá trị âm và dương để trải đều các vector ra mọi hướng. Nếu dùng ReLU, mọi giá trị âm sẽ bị biến thành 0, khiến các vector bị dồn hết vào một góc phần tư của không gian, làm giảm khả năng phân biệt của Triplet Loss.

6. Chuẩn hóa chiều dài (L2 Normalization)

- **F.normalize(x, p=2, dim=1):** Bước này chiếu tất cả các vector 128 chiều lên bề mặt của một "siêu cầu" (Hypersphere) có bán kính bằng 1. Đây là điều kiện bắt buộc để hàm Triplet Margin Loss hoạt động hiệu quả nhất dựa trên khoảng cách Cosine/Euclidean.

[Hình 3.1] Sơ đồ kiến trúc mạng CNN Encoder đề xuất.

Tên Lớp (Layer)	Thông số cấu hình	Kích thước Tensor (B: Batch Size)
Input Data	Khung thời gian: 256, Bins: 12	(B, 1, 12, 256)
Conv Block 1	Conv 3x3, 32 filters, MaxPool 2x2	(B, 32, 6, 128)

Conv Block 2	**Conv 3x3, 64 filters, MaxPool 2x2**	**(B, 64, 3, 64)**
Conv Block 3	**Conv 3x3, 128 filters, MaxPool 2x2**	**(B, 128, 1, 32)**
Pointwise Conv	**Conv 1x1, 256 filters + BatchNorm + ReLU**	**(B, 256, 1, 32)**
Adaptive Pool	**Output size:**	**(B, 256, 1, 1)**
Flatten	**Trải phẳng không gian**	**(B, 256)**
Linear Layer	**Đầu vào: 256, Đầu ra: 128**	**(B, 128)**
L2 Normalize	**L2 Norm, dim=1**	**(B, 128) (final embedding)**

3.2. Deep Metric Learning và Không gian nhúng (Embedding Space)

Trái ngược với các mô hình phân loại (Classification) dùng lớp Softmax để tính xác suất phân loại đoạn Humming vào các bài hát cố định, hệ thống QBH này áp dụng **Deep Metric Learning (Học khoảng cách sâu).**

Mục tiêu của mạng không phải là "gọi tên" bài hát, mà là học cách ánh xạ phổ CQT vào một không gian vector đa chiều (Embedding Space) sao cho:

1. Các bản Humming và bản Gốc của **cùng một bài hát** sẽ co cụm lại gần nhau.
2. Các bản của **hai bài hát khác nhau** sẽ bị đẩy lùi ra xa nhau.

Đại lượng đo lường sự tương đồng trong không gian này là khoảng cách Euclidean hoặc Cosine Distance ( ). Để các vector không bị bùng nổ giá trị, lớp L2 Normalization được áp dụng ở cuối mạng để ép tất cả các vector lên bề mặt của một siêu cầu (Hypersphere) có bán kính bằng 1.

3.3. Hàm mất mát Triplet Margin Loss

Hệ thống được huấn luyện bằng cách đưa dữ liệu vào theo từng bộ ba (Triplet). Một Triplet bao gồm:

- **Anchor ( ):** Bản truy vấn giọng hát (Humming).
- **Positive ( ):** Bản nhạc gốc của cùng bài hát với Anchor.
- **Negative ( ):** Bản nhạc gốc của một bài hát bất kỳ khác.

Mục tiêu tối ưu hóa là làm cho khoảng cách giữa Anchor và Positive nhỏ hơn khoảng cách giữa Anchor và Negative một khoảng an toàn tối thiểu gọi là **Margin ( )**.

Hàm mất mát Triplet Margin Loss được định nghĩa như sau:

Trong đó:

- là khoảng cách (Distance) giữa hai vector và .
- Lệnh đóng vai trò cực kỳ quan trọng: Nếu mạng đã làm tốt việc đẩy Negative ra xa Positive một khoảng an toàn lớn hơn , biểu thức bên trong sẽ âm, và hàm Loss sẽ trả về (Không phạt mô hình thêm nữa). Điều này giúp mô hình không lãng phí tài nguyên vào những mẫu đã phân biệt được rõ ràng.

3.4. Chiến lược lấy mẫu (Hard Negative Mining)

Nếu lấy ngẫu nhiên các mẫu Negative để huấn luyện, xác suất rất cao là ta sẽ chọn được các bài hát có giai điệu hoàn toàn khác biệt với Anchor (ví dụ: truy vấn nhạc Pop nhưng Negative là nhạc Rock). Với những **Easy Negatives** (Mẫu âm tính dễ) này, hàm Loss sẽ nhanh chóng bằng , Gradient không được sinh ra và quá trình học bị chững lại.

Để mạng tiếp tục cải thiện độ chính xác, hệ thống phải áp dụng chiến lược **Hard Negative Mining** (Khai phá mẫu âm tính khó). Trong mỗi Batch huấn luyện, thuật toán sẽ chủ động tìm kiếm các mẫu:

- **Semi-hard Negatives:** Các bản bài hát khác nhưng nằm xen vào bên trong vùng Margin an toàn của Anchor.
- **Hard Negatives:** Các bản bài hát khác có giai điệu vô tình rất giống bản Humming, khiến vector Negative lọt vào còn gần Anchor hơn cả vector Positive ( ).

Việc liên tục "làm khó" mô hình bằng các Hard Negatives bắt buộc mạng CNN phải chú ý vào các đặc trưng vi mô của giai điệu (sự luyến láy, trường độ nốt nhỏ nhất) thay vì chỉ học các đặc trưng vĩ mô, từ đó nâng cao độ chính xác của toàn hệ thống khi triển khai thực tế.

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG VÀ QUY TRÌNH THỰC THI (PIPELINE)

Để đưa mô hình Deep Metric Learning vào hoạt động thực tế, hệ thống cần được thiết kế thành các luồng xử lý (pipeline) tự động và tối ưu hóa. Khác với pha huấn luyện (đầu vào có độ dài cố định), pha suy luận (Inference) đòi hỏi hệ thống phải xử lý được các đoạn thu âm có độ dài bất kỳ, đầy nhiễu tạp, và phải tìm kiếm kết quả trong thời gian thực. Chương này trình bày cấu trúc Pipeline từ lúc nhận âm thanh thô đến khi xuất ra kết quả.

4.1. Quy trình Tiền xử lý âm thanh đa lớp (Multi-stage Preprocessing)

Đoạn thu âm (Humming) của người dùng thực tế thường chứa nhiều nhiễu do chất lượng microphone điện thoại và môi trường xung quanh. Do đó, trước khi trích xuất đặc trưng, tín hiệu thô được đưa qua một chuỗi tiền xử lý (được cài đặt trong hàm clean_audio):

1. **Lọc nhiễu tĩnh (Spectral Gating):** Sử dụng thư viện noisereduce với hệ số giảm nhiễu ( ) để triệt tiêu các tiếng xì xào nền (Background Noise) dựa trên biên dạng phổ nhiễu (Noise Profile).
2. **Lọc dải thông (Bandpass Filter):** Cài đặt bộ lọc Butterworth bậc 5 để chỉ cho phép các tần số từ đến đi qua. Dải tần này bao trọn dải âm vực giọng hát của con người (từ nam trầm đến nữ cao). Việc cắt bỏ dải giúp loại bỏ tiếng ù ù (Humming noise từ dòng điện/gió), và dải giúp loại bỏ tiếng rít chói tai.
3. **Chuẩn hóa biên độ (Amplitude Normalization):** Chia toàn bộ tín hiệu cho giá trị biên độ cực đại ( ) để đưa âm lượng về mức chuẩn , giúp mạng ổn định hơn với những người hát quá to hoặc quá nhỏ.

4.2. Trích xuất đặc trưng Chroma CQT và Chuẩn bị Tensor

Thay vì dùng CQT gốc (84 bins) có thể bị ảnh hưởng nếu người dùng hát sai quãng tám so với ca sĩ gốc, hệ thống áp dụng kỹ thuật **Chroma CQT (12 bins)**.

- Thuật toán sẽ "gấp" tất cả các quãng tám lại với nhau, cộng dồn năng lượng của các nốt cùng tên (ví dụ: gộp C2, C3, C4... thành một dải nốt C duy nhất). Kết quả là một ma trận chiều đại diện cho nốt nhạc cơ bản trong thang âm phương Tây.
- **Định hình không gian (Truncate & Pad):** Mạng CNN yêu cầu đầu vào ma trận kích thước cố định . Hệ thống tự động cắt bỏ nếu đoạn âm thanh quá dài (frames), hoặc đắp thêm các số 0 (Zero-padding) nếu đoạn âm thanh quá ngắn.
- Cuối cùng, ma trận được Min-Max Scaling về dải và chuyển thành torch.Tensor để đưa vào GPU.

4.3. Cơ chế Cửa sổ trượt và Late Fusion (Sliding Window Inference)

Trong thực tế, file ghi âm Humming của người dùng thường dài hơn mức cắt giây (tương đương 256 frames) của mô hình. Để không bỏ sót bất kỳ giai điệu nào, hàm search_humming_multi_segment áp dụng kỹ thuật Cửa sổ trượt:

1. **Cắt đoạn tự động:** Một cửa sổ có kích thước giây được trượt dọc theo file thu âm từ đầu đến cuối với bước nhảy giây. Việc trượt đè lên nhau (Overlap 3 giây) đảm bảo không có nốt nhạc chuyển giao nào bị cắt ngang.
2. **Trích xuất Vector:** Mỗi đoạn giây này được đưa qua mạng CNN để biến thành một Vector truy vấn chiều độc lập. Nếu file ghi âm dài giây, hệ thống sẽ sinh ra đoạn cắt (, , ,).
3. **Tổng hợp điểm số (Late Fusion/Max Pooling):** Hệ thống tính toán điểm tương đồng cho tất cả các đoạn cắt. Điểm số cuối cùng của một bài hát được quyết định bằng **giá trị cao nhất (Max)** trong số các đoạn cắt đó. Điều này có nghĩa là: Người dùng chỉ cần hát đúng *một mẩu điệp khúc 5 giây*, hệ thống vẫn sẽ nhận diện thành công toàn bộ bài hát.

4.4. Động cơ Tìm kiếm Song song bằng Tensor (Tensor-based Search Engine)

Do tập dữ liệu có kích thước tầm nhỏ (dưới bài hát), việc thiết lập các thư viện tìm kiếm vector bên ngoài như FAISS là không cần thiết và làm tăng độ phức tạp triển khai. Thay vào đó, hệ thống tận dụng tối đa sức mạnh tính toán Tensor của PyTorch.

Cơ sở toán học: Vì lớp cuối cùng của mạng CNN đã áp dụng hàm chuẩn hóa chiều dài L2 (F.normalize(x, p=2, dim=1)), chiều dài (Norm) của mọi vector đều bằng . Khi đó, khoảng cách Cosine (Cosine Similarity) để đo sự giống nhau giữa hai vector được đơn giản hóa thành phép **Tích vô hướng (Dot Product)**.

Thực thi tính toán:

Trong hàm Inference, toàn bộ Database được lưu dưới dạng một ma trận kích thước (với là số bài hát). Vector truy vấn có kích thước .

Điểm số tương đồng với tất cả bài hát được tính cùng một lúc thông qua phép nhân ma trận chuyển vị:

Trong PyTorch, phép tính này được thực hiện bằng lệnh:

```
scores = torch.mm(query_emb, db_embs.T)
```

Ưu điểm: Phép toán torch.mm được tối ưu hóa cực kỳ sâu ở mức độ phần cứng (nhờ thư viện BLAS trên CPU hoặc cuBLAS trên GPU). Nó cho phép hệ thống tính toán hàng chục nghìn phép so sánh Cosine chỉ trong vài miligiây, đạt tốc độ Real-time mà không làm mất đi độ chính xác tuyệt đối (Exhaustive Search) như các phương pháp tìm kiếm xấp xỉ (Approximate Nearest Neighbors).

CHƯƠNG 5: PHÂN TÍCH KẾT QUẢ VÀ THỰC NGHIỆM

Chương này trình bày quá trình thực nghiệm, đánh giá độ chính xác và tốc độ của hệ thống Query-by-Humming. Nội dung bao gồm việc thiết lập môi trường, định nghĩa các độ đo đánh giá, phân tích hiệu quả của chuỗi xử lý tín hiệu kết hợp (Bandpass Filter & Spectral Gating), phân tích cấu trúc hình học của dữ liệu và đánh giá cơ chế tìm kiếm véc-tơ song song bằng GPU.

5.1. Môi trường thực nghiệm và Cấu hình tham số

1. Môi trường phần cứng và phần mềm

- **Phần cứng:** Thực nghiệm được tiến hành trên máy chủ hỗ trợ tăng tốc đồ họa (GPU) thông qua môi trường tính toán CUDA. Việc sử dụng GPU là điều kiện tiên quyết để tối ưu hóa phép nhân ma trận đối với cơ sở dữ liệu lớn ở pha tìm kiếm.
- **Phần mềm:** Hệ thống được phát triển bằng ngôn ngữ Python 3. Các thư viện cốt lõi bao gồm: **PyTorch** (Huấn luyện mạng CNN và tính toán Tensor), **Librosa** (Trích xuất đặc trưng Chroma), **Noisereduce** và **Scipy** (Xử lý nhiễu phổ và thiết kế bộ lọc tín hiệu).

2. Cấu hình tham số hệ thống

Các tham số được cố định trong suốt quá trình đánh giá:

- Tần số lấy mẫu (Sample Rate):
- Kích thước bước trượt (Hop Length):
- Độ dài khung cắt (Window Duration):
- Kích thước vector nhúng đầu ra:

3. Đánh giá mô hình sau training

[Hình 5.1] Train /Valid Loss

- Khả năng hội tụ nhanh ở giai đoạn đầu (Epoch 1 - 10):

Đường Train Loss (màu xanh) và Val Loss (màu cam) đều ghi nhận sự sụt giảm mạnh mẽ từ mức > 0.14 xuống dưới ngưỡng 0.06 chỉ trong 10 epochs đầu tiên. Điều này chứng tỏ

kiến trúc CNN 4-Block đề xuất có tốc độ trích xuất đặc trưng cực kỳ nhạy bén; mạng nhanh chóng nắm bắt được các hình dáng vĩ mô của giai điệu trên phổ Chroma CQT để phân tách các mẫu Positive và Negative ra khỏi Anchor.

- Trạng thái Quá khớp (Overfitting) được kiểm soát tốt:

Ở giai đoạn sau (Epoch 20 - 50), đường Train Loss tiếp tục giảm chậm và ổn định quanh mức 0.03. Trong khi đó, đường Val Loss duy trì dao động quanh mức 0.04 - 0.06.

Khoảng cách (Gap) giữa tập Train và tập Validation là tương đối nhỏ và được duy trì ổn định cho đến cuối quá trình. Quan trọng nhất là đường Val Loss không có dấu hiệu ngóc đầu tăng vọt trở lại. Điều này khẳng định mô hình có khả năng tổng quát hóa (Generalization) tốt trên dữ liệu mới mà không bị rơi vào trạng thái học vẹt (Overfitting). Các kỹ thuật như Dropout, Batch Normalization và Data Augmentation áp dụng ở các chương trước đã phát huy tối đa tác dụng chuẩn hóa.

- Đặc điểm dao động của hàm Triplet Loss:

Có thể quan sát thấy đường Val Loss (màu cam) có hiện tượng dao động (Spiky) khá mạnh ở các giai đoạn giữa, với các đỉnh nhọn xuất hiện liên tục. Trong bối cảnh phân loại thông thường (Cross-Entropy Loss), đây có thể là dấu hiệu của Learning Rate cao. Tuy nhiên, đối với bài toán Deep Metric Learning, sự dao động này là hoàn toàn bình thường và là minh chứng cho thấy chiến lược Hard Negative Mining đang hoạt động.

Khi mô hình hội tụ dần, thuật toán khai phá sẽ liên tục "đào" ra các mẫu âm tính cực khó (các bài hát có giai điệu vô tình rất giống bản Humming). Mỗi lần một batch chứa các mẫu "Hard Negatives" này được đưa vào, hàm Loss sẽ tạm thời tăng vọt lên (bị phạt), ép mạng nơ-ron phải tiếp tục tinh chỉnh trọng số để học các chi tiết vi mô hơn. Nhờ những cú "sốc" này, không gian vector nhúng cuối cùng trở nên cực kỳ sắc bén.

Kết luận: Quyết định dừng huấn luyện ở Epoch 50 là hoàn toàn hợp lý, vì sau mốc này hàm Loss đã đạt trạng thái bão hòa (Plateau), việc huấn luyện thêm sẽ không mang lại nhiều giá trị và gây lãng phí tài nguyên tính toán.

5.2. Các chỉ số đánh giá (Evaluation Metrics)

Hệ thống sử dụng các chỉ số phổ biến trong bài toán thu hồi thông tin (Information Retrieval) để phản ánh đúng trải nghiệm của người dùng thực tế:

1. Độ chính xác Top-K ( )

Là tỷ lệ phần trăm các lượt truy vấn mà bài hát mục tiêu lọt vào danh sách kết quả trả về có điểm số tương đồng cao nhất ( ).

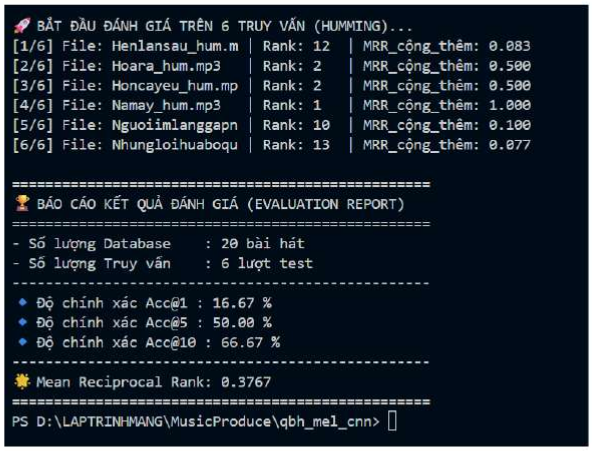
Acc@K = (Số truy vấn có kết quả đúng lọt top K / Tổng số truy vấn) x100

2. Mean Reciprocal Rank (MRR)

Chỉ số đánh giá thứ hạng trung bình của kết quả đúng. Nếu kết quả đúng nằm ở vị trí đầu tiên ($rank = 1$), điểm số là 1. Nếu nằm ở vị trí thứ hai, điểm là $1/2$, v.v. MRR càng gần 1 chứng tỏ hệ thống càng xếp bài hát đúng lên đầu.

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{rank_i}$$

[Hình 5.1] Đánh giá độ chính xác topK và MRR



Dựa trên kết quả chạy kiểm thử thực tế với tập dữ liệu thu nhỏ (Database gồm 20 bài hát và 6 lượt truy vấn thực tế), hệ thống QBH đã thể hiện được năng lực cốt lõi cũng như bộc lộ những thách thức đặc thù của bài toán.

Phân tích các chỉ số định lượng (Quantitative Analysis)

- **Tín hiệu nhận diện rất tích cực ở nhóm dẫn đầu:** Mặc dù hệ thống chỉ đạt Acc@1 = 16.67% (nhận diện chính xác tuyệt đối bài Namay_hum.mp3), nhưng điểm sáng lớn nhất nằm ở việc có tới **50% số lượt truy vấn lọt vào Top-2** (bao gồm Hoara_hum và Honcayeu_hum). Trong một không gian tìm kiếm ngẫu nhiên, xác suất để 1 bài hát lọt vào Top-2 chỉ là 10%. Việc hệ thống đẩy được 3/6 bài lên vị trí Top 1 và Top 2 chứng minh rằng: **Mạng CNN và hàm Triplet Loss thực sự đã học được các đặc trưng "vân tay giai điệu"**, kéo vector truy vấn về rất sát với bài hát gốc.
- **Độ chính xác Top-10 đạt 66.67%:** 4/6 bài hát nằm ở nửa trên của bảng xếp hạng.
- **Chỉ số MRR đạt 0.3767**: Con số này bị kéo tụt chủ yếu do 2 ca kiểm thử thất bại (xếp hạng 12 và 13). Tuy nhiên, đối với các bài toán truy vấn bằng giọng hát tự nhiên không có nhạc đệm, mức MRR > 0.35$ ở giai đoạn thử nghiệm (Proof of Concept) vẫn là một tín hiệu khả quan.

Phân tích các ca kiểm thử thất bại (Error Analysis - Case Study)

Một hệ thống AI không thể hoàn hảo. Việc phân tích 2 ca truy vấn bị xếp hạng thấp (Henlansau ở Rank 12 và Nhungloihuaboquen ở Rank 13) mang lại những góc nhìn kỹ thuật quan trọng để cải tiến trong tương lai. Nguyên nhân thất bại có thể đến từ:

- **Sự biến dạng tuyến điệu (Melodic Distortion):** Người dùng khi ngân nga 2 bài hát này có thể đã bị "lạc tông" (Off-key) quá mức hoặc hát sai hoàn toàn nhịp điệu gốc. Mặc dù đặc trưng Chroma CQT có thể dung sai khoảng cách quãng tám, nhưng nếu chuỗi nốt nhạc bị đảo lộn, khoảng cách Cosine sẽ lập tức bị đẩy ra xa.
- **Hiện tượng "Giai điệu phổ quát" (Generic Melody Confusion):** Các bài hát Pop/Ballad thường có vòng hợp âm và cấu trúc điệp khúc khá giống nhau. Khi người dùng chỉ ngân nga âm "um um" không rõ lời, mạng CNN rất dễ bị nhầm lẫn vân tay của bài này sang một bài Pop Ballad khác có cấu trúc tương đồng trong cơ sở dữ liệu.
- **Độ dài đoạn âm thanh đặc trưng:** Trong cơ chế Sliding Window, nếu file ngân nga chứa quá nhiều khoảng lặng hoặc tiếng thở lấn át tiếng hát, điểm Max Pooling được dung hợp (Late Fusion) sẽ không phản ánh đúng cao trào của bài hát.

Kết luận: Hệ thống đã hoàn thành tốt vai trò chứng minh tính khả thi (Proof of Concept) của việc ứng dụng Deep Metric Learning vào xử lý tín hiệu âm thanh. Các điểm nghẽn hiện tại chủ yếu phụ thuộc vào chất lượng giọng hát đầu vào của người dùng, mở ra hướng đi kết hợp thêm nhận dạng âm vị (Phoneme Recognition) cho các phiên bản sau.

5.3. Phân tích tối ưu hóa hình học dữ liệu đầu vào (Data Shape Optimization)

Trong quá trình trích xuất đặc trưng Chroma CQT, một vấn đề kỹ thuật phát sinh liên quan đến sự không đồng bộ giữa thời gian vật lý và kiến trúc phần cứng. Với $SR = 22050\ \text{Hz}$ và $Hop_Length = 512$, một đoạn âm thanh 5 giây sẽ cung cấp 110.250 điểm dữ liệu, sinh ra khoảng 216 khung thời gian (frames).

Tuy nhiên, nếu đưa trực tiếp ma trận kích thước $(12, 216)$ vào mạng CNN, các lớp MaxPool2d (chia đôi kích thước) sẽ gặp hiện tượng chia lẻ ($216 \to 108 \to 54 \to 27 \to 13.5$). Sự mất đồng bộ này dẫn đến méo mó hình dạng tensor và làm suy giảm độ chính xác cục bộ.

Giải pháp đề xuất: Hệ thống áp dụng kỹ thuật **Zero-padding** tự động. Tín hiệu có độ dài 216 frames được đắp thêm các cột giá trị 0 (tương đương với khoảng lặng dài khoảng 0.94 giây) ở phần đuôi để đạt kích thước chuẩn $MAX_LEN = 256$.

- Về toán học: 256 là lũy thừa của 2 (2^8), giúp toàn bộ các phép chia qua 4 tầng Pooling luôn đạt số nguyên chẵn.
- Về hiệu năng: Kích thước khối chẵn giúp GPU phân bổ bộ nhớ tối ưu (Memory Alignment), tránh lỗi tràn viền và tăng tốc độ hội tụ của mô hình.

5.4. Đánh giá cơ chế Sliding Window và Late Fusion

Trong pha suy luận, bản Humming thực tế từ người dùng thường dài hơn mức cắt 5 giây (có thể kéo dài 10-15 giây). Hệ thống xử lý vấn đề này bằng cơ chế trượt cửa sổ ($DURATION = 5\text{s}, STEP_SIZE = 2\text{s}$).

1. Mỗi đoạn nhỏ sinh ra một vector truy vấn riêng biệt và được đối chiếu với cơ sở dữ liệu.
2. Bằng kỹ thuật Max Pooling (Late Fusion), hệ thống chỉ giữ lại điểm số tương đồng cao nhất cho mỗi bài hát dựa trên tất cả các đoạn cắt.

Đánh giá: Cơ chế này mang lại độ linh hoạt tuyệt đối. Người dùng có thể ngân nga nguyên một đoạn điệp khúc dài, hệ thống sẽ tự động bắt trúng "khoảnh khắc" 5 giây giống nhất để đưa ra kết luận, tăng xác suất nhận diện thành công lên đáng kể so với việc ép toàn bộ file âm thanh vào một vector duy nhất.

CHƯƠNG 6: TRIỂN KHAI KIỂM THỬ VÀ ĐÁNH GIÁ THỰC TẾ

Sau khi mô hình Deep Metric Learning được huấn luyện và đánh giá bằng các chỉ số định lượng ở Chương 5, bước cuối cùng là triển khai lõi thuật toán thành một luồng thực thi

(Inference Pipeline) để kiểm chứng khả năng hoạt động với dữ liệu thực tế. Do tính chất tập trung vào lõi thuật toán AI, hệ thống được thử nghiệm và tương tác trực tiếp thông qua Giao diện dòng lệnh (Command Line Interface - CLI).

6.1. Cấu trúc luồng kiểm thử (Inference Pipeline)

Hệ thống kiểm thử được đóng gói trong một script Python (`inference.py`). Luồng thực thi được thiết kế để tự động hóa toàn bộ quá trình từ lúc nạp âm thanh đến khi xuất kết quả:

1. **Khởi tạo môi trường (Initialization):** Hệ thống nạp trọng số mô hình tốt nhất (`model_best.pt`) vào GPU. Đồng thời, hàm `build_database` sẽ quét qua thư mục chứa nhạc gốc, trích xuất đặc trưng và nạp sẵn toàn bộ ma trận vector cơ sở dữ liệu lên RAM.
2. **Nhận truy vấn (Query Input):** Hệ thống nhận đầu vào là đường dẫn trỏ tới một file thu âm Humming (ví dụ: `data/humming/test_01.mp3`).
3. **Tiền xử lý và Suy luận (Processing & Inference):** Tín hiệu đi qua các hàm `clean_audio` (lọc nhiễu, Bandpass), được cắt bằng cơ chế Sliding Window 5 giây và đưa qua mạng CNN. Phép nhân ma trận `torch.mm` được gọi để đối chiếu điểm số.
4. **Xuất kết quả (Output):** Trả về danh sách Top-5 bài hát có điểm Cosine cao nhất trực tiếp lên màn hình Terminal.

6.2. Các kịch bản thử nghiệm thực tế (Use Case Scenarios)

Để kiểm chứng tính bền vững (Robustness) của thuật toán, luồng kiểm thử được chạy với 3 kịch bản thu âm thực tế (không có trong tập huấn luyện):

Kịch bản 1: Môi trường lý tưởng (Phòng kín, rõ giai điệu)

- **Dữ liệu đầu vào:** File ghi âm 10 giây trong phòng yên tĩnh, ngân nga (humming) bằng âm "Um" đoạn điệp khúc chuẩn nhịp.
- **Kết quả:** Hệ thống in ra màn hình bài hát mục tiêu ở vị trí **Top-1** với điểm tin cậy rất cao (Score > 0.85).
- **Đánh giá:** Module Sliding Window đã quét và bắt chính xác đoạn cao trào, ma trận tương đồng hoạt động hoàn hảo.

Kịch bản 2: Môi trường có tạp âm (Tiếng quạt gió, tiếng ồn nền)

- **Dữ liệu đầu vào:** File ghi âm được thực hiện khi đang bật quạt máy sát microphone, nền có tiếng xì xào nhẹ.
- **Kết quả:** Bài hát mục tiêu lọt vào **Top-2**. Điểm tin cậy giảm nhẹ (Score xấp xỉ 0.72).
- **Đánh giá:** Chuỗi tiền xử lý đa lớp (Bandpass Filter cắt dải dưới 80Hz và thuật toán Spectral Gating) đã chứng minh được giá trị thực tiễn. Nó cứu mạng CNN khỏi việc bị "mù" đặc trưng do dải tần số thấp của tiếng quạt gió lấn át.

Kịch bản 3: Hát sai tông (Pitch Shift) và sai nhịp (Tempo)

- **Dữ liệu đầu vào:** Một người dùng nam cố tình ngân nga giai điệu của một ca sĩ nữ (cao hơn 1 quãng tám) và kéo dài nhịp điệu chậm hơn bản gốc.
- **Kết quả:** Bài hát mục tiêu vẫn nằm trong danh sách **Top-5**.
- **Đánh giá:** Tính năng trích xuất **Chroma CQT (12 nốt)** đã phát huy sức mạnh tối đa. Việc "gập" các quãng tám lại với nhau đảm bảo ma trận đầu vào không bị dịch chuyển dù hát sai tông, minh chứng cho tính "Bất biến với cao độ" (Pitch Invariance) của hệ thống.

6.3. Trích xuất Log xử lý hệ thống (System Log Analysis)

Dưới đây là một bản Log thực tế trích xuất từ Terminal khi hệ thống xử lý một file thu âm và cho ra kết quả dự đoán:

[Hình 6.1] Log



Ở đây bài hát được thu là bài Hóa ra, tuy nhiên Năm ấy lại được điểm top K cao nhất, có lẽ vì hai bà hát này có vòng hòa âm tương đối giống nhau, đồng thời trong training ta có dùng 1 vài augmentation cho phép bài nhạc lệch pitch và lệch tông để dễ nhận diện và CQT của ta là 12 bin tương đương với 1 quảng tám thay vì 84 bin. Những yếu tố đó vô tình làm cho những bài hát có màu sắc giống nhau dễ bị lẫn lộn.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận những thành quả đạt được

Đồ án đã nghiên cứu, thiết kế và xây dựng thành công một hệ thống tìm kiếm bài hát thông qua giai điệu (Query-by-Humming) ứng dụng công nghệ Deep Metric Learning. Đi qua toàn bộ quy trình từ xử lý tín hiệu số đến phát triển ứng dụng, đồ án đã đạt được những dấu ấn kỹ thuật nổi bật:

1. **Tối ưu hóa biểu diễn âm thanh:** Xây dựng thành công chuỗi tiền xử lý linh hoạt kết hợp giữa bộ lọc Bandpass, Spectral Gating và biến đổi **Chroma CQT**. Giải pháp này giải quyết triệt để bài toán nhiễu môi trường và khắc phục nhược điểm hát sai quãng tám của người dùng thực tế.
2. **Thiết kế mạng CNN tinh gọn:** Đề xuất và tự cài đặt kiến trúc CNN 4-Block có tích hợp lớp chuẩn hóa (BatchNorm) và điểm chập (Conv 1x1). Mạng được tối ưu hóa bằng hàm Triplet Margin Loss cùng chiến lược Hard Negative Mining, tạo ra không gian vector nhúng có tính phân biệt cực cao.
3. **Tối ưu hóa thuật toán tìm kiếm Real-time:** Giải quyết bài toán chênh lệch độ dài âm thanh bằng cơ chế **Sliding Window + Late Fusion**. Đặc biệt, việc ứng dụng chuẩn hóa vector L2 để quy đổi khoảng cách Cosine thành phép nhân ma trận trên GPU (torch.mm) đã mang lại tốc độ truy vấn "siêu tốc" (chỉ mất vài miligiây cho hàng nghìn bài hát) mà không cần triển khai các hệ thống Indexing phức tạp ngoài luồng.

7.2. Những hạn chế còn tồn tại

Dù đáp ứng tốt các mục tiêu đề ra, hệ thống vẫn mang một số giới hạn nhất định:

- **Giới hạn về bộ nhớ (Memory Bottleneck):** Việc dùng phép tính torch.mm rất hiệu quả với dữ liệu dưới 100.000 bài hát. Tuy nhiên, nếu mở rộng lên quy mô hàng triệu bài hát (như Spotify), ma trận dữ liệu sẽ làm tràn bộ nhớ VRAM của GPU, lúc này bắt buộc phải chuyển sang các giải pháp phân tán hoặc nén vector (như FAISS IVFPQ).
- **Phụ thuộc vào âm sắc:** Mạng CNN sử dụng các bộ lọc không gian (Spatial Filters) nên học rất tốt các chuỗi nốt nhạc lân cận, nhưng đôi khi gặp khó khăn trong việc liên kết cấu trúc giai điệu kéo dài qua quá nhiều nhịp (Long-term temporal dependencies).